Highlights

Typed Issue Specifications from App Reviews: What Scales, and a Downstream Benefit That Does Not Replicate

Fabiha Jalal, Sadia Tasnim Dhruba, Tazrian Zaman Tushti, Ajwad Abrar, Md Kamrul Hasan, Hasan Mahmud

- Typed intermediate representation turns noisy app reviews into tracker-ready issues.
- A verified-anchor procedure lifts Cohen's kappa from 0.163 to 0.592 at corpus scale.
- Template-field fill rate reaches 0.96 against 0.69 without type routing.
- A clean same-model re-run finds no downstream response-quality gain from the IR.
- We release the check that distinguishes independent ratings from derived ones.

Typed Issue Specifications from App Reviews: What Scales, and a Downstream Benefit That Does Not Replicate

Fabiha Jalal[a,*], Sadia Tasnim Dhruba[a], Tazrian Zaman Tushti[a], Ajwad Abrar[b], Md Kamrul Hasan[b], Hasan Mahmud[b]

[a]*Department of Computer Science, Islamic University of Technology (IUT), Gazipur, Bangladesh*
[b]*Systems and Software Lab (SSL), Department of Computer Science and Engineering, Islamic University of Technology (IUT), Gazipur, Bangladesh*

Abstract

Context. Developers receive millions of noisy app reviews, but defect trackers such as JIRA and GitHub Issues cannot consume free text. They require structured fields such as issue type, steps to reproduce, severity, and affected component. Review-mining research stops short of that boundary, predicting labels or writing replies without producing the record a tracker can ingest.

Objective. Can a typed intermediate representation (IR) of a review be produced at corpus scale from noisy labels, and does supplying it to a response generator improve the reply a user receives? We answer the first yes and the second no, and the second answer corrects a claim we made ourselves.

Method. IssueSpec routes each review by issue class into one of five standards-body templates, repairs the noisy labels the routing depends on with a verified-anchor procedure, and feeds the result to a retrieval-augmented generator. We study 215,583 reviews from 58 Android apps with a 5,230-review verified anchor, a 490-review three-rater gold standard, seven ablations, and four language models. The downstream evaluation was run

*Corresponding author.

Email addresses: `fabihajalal@iut-dhaka.edu` (Fabiha Jalal), `sadiatasnim@iut-dhaka.edu` (Sadia Tasnim Dhruba), `tazrian@iut-dhaka.edu` (Tazrian Zaman Tushti), `ajwadabrar@iut-dhaka.edu` (Ajwad Abrar), `hasank@iut-dhaka.edu` (Md Kamrul Hasan), `hasan@iut-dhaka.edu` (Hasan Mahmud)

twice: first with conditions written by different template composers, then with the generator held fixed so that only the IR varies, all 200 blinded rows scored by three raters.

Results. Production works. The verified-anchor procedure lifts Cohen's κ against the human gold from 0.163 to 0.592, and to 0.616 on reviews the classifier never saw. Type routing reaches 0.96 template-fill against 0.69 without it, a schema-conformance measure rather than a quality one. The downstream benefit does not. The first round measured +2.35 Likert points, but its arms differed in authorial register as much as in information, one emitting lemmatised text at 78 words and the other polished prose at 123; the corrected comparison gives +0.04 ($p = 0.38$), with raters disagreeing in direction. Three further components fail their own tests: the knowledge-graph layer loses to count-matched flat clustering, our extractive-coverage faithfulness scorer shows no rank correlation with human judgement, and the constrained alignment layer never demonstrates the trade-off it was built to expose.

Conclusions. Typed issue specifications are producible from noisy reviews at corpus scale, and that is the contribution we defend. Their claimed downstream payoff is not. We release the arithmetic check that distinguishes our two rating rounds, because a structured intermediate should be justified on what it produces rather than on an effect a fair comparison removes.

Keywords: App review mining, Issue report generation, Intermediate representation, Knowledge graph, Retrieval-augmented generation, Reinforcement learning from human feedback, Empirical software engineering

---

1. Introduction

App stores are where users talk back to developers. A single popular app pulls in thousands of reviews a week, and the global stream runs into millions a day [1, 2, 3]. These reviews carry the signals developers need to act on: bug reports (a crash on Samsung S8 after an update), feature requests (dark mode), performance complaints (battery drain), and regression notes (voice search no longer works). The information is real, but it shows up as messy free-form text, usually without the details required to reproduce a bug or scope a new feature.

To turn any of this into work, developers rely on defect trackers such as

JIRA or GitHub Issues. Those tools, though, can't work with raw text; they need structured fields such as issue type, steps to reproduce, severity, and affected component [4, 5]. Filling those fields by hand does not scale with the volume modern apps face. Years of work on automated review pipelines has helped with the surrounding tasks like classification, summarization, and response generation, but no system actually produces the structured report a developer can paste directly into a tracker. Dąbrowski et al.'s nineteen-year survey of the field [3] confirms the same boundary.

Prior work has tackled several pieces of this problem separately. Classifiers [1, 6, 7, 8] take a review and predict an issue label such as bug or feature request. Response generators [2, 9, 10] produce a polite reply for the user. Agentic software-engineering tools such as SWE-Agent [11] and RepairAgent [12] take a clean GitHub issue and generate a code patch. Knowledge-graph methods [13, 14, 15] cluster reviews into themes or surface frequent complaints. Each of these has matured over the past decade and works well within its own scope.

But none of these reaches the developer-ready issue report a defect tracker needs. The Maalej-Nabil classifier taxonomy [1] and its successors [6, 7, 8, 16, 17] have settled at an F1 ceiling around 0.75–0.85, a label-quality limit (not a model-capacity one) per Dąbrowski et al.'s [3] nineteen-year survey. The label is only the first piece of what a tracker entry needs; the remaining fields (steps to reproduce, severity, affected component) have to come from somewhere, and classification-only pipelines leave that downstream step to manual triage. Response generators like RRGen [2] and CoRe [9] address the customer-communication side of the workflow, but the actual bug behind the review is never written out in the structured form a tracker needs, so even a perfect reply system does not give developers something they can paste into JIRA. Agentic SE tools assume a well-formed bug report or GitHub issue as the agent's starting state, built into the SWE-bench protocol [18] where each instance is a (GitHub issue, code repository) pair; constructing such an issue from raw review text is explicitly outside the protocol, leaving the upstream gap unaddressed.

Knowledge-graph methods stop at intermediate outputs rather than typed fields: ReviewGraph [13], KGCPN [14], and Keertipati et al. [15] each output rating predictions, summaries, or feature rankings, none of which a defect tracker can ingest directly. RLHF has matured into the standard approach for aligning LLM outputs with human preferences (DPO [19], KTO [20], PPO [21]), and Safe-RLHF [22] extends the framework to multi-objective settings under a Constrained MDP for general LLM safety (harmfulness, dishonesty, bias). The developer-relations setting poses a different question entirely: how to align responses under a domain-specific constraint family (truthful promises, no information leakage, professional tone, legal-safe wording) rooted in business and legal exposure. To our knowledge, no published RLHF formulation has been adapted to this constraint set.

This work pursues three aims. First, we develop a framework that turns noisy app reviews into structured, taxonomy-grounded issue specifications a defect tracker can consume directly. Second, we build a response generator that consumes these specifications, and we test whether doing so changes the reply a user receives; Section 5.2 reports that it does not measurably. Third, we build a dual-objective alignment trainer intended to optimise response helpfulness under the domain-specific compliance constraints of the developer-relations setting, and we test whether the two objectives can be shown to trade off; Section 5.3 reports that at the scale we can train, they cannot.

These aims map to three research questions.

RQ1 (Translation Quality). Can a typed IR, generated via a three-layer framework (knowledge-graph construction, hierarchical clustering, standardized schema mapping), produce issue specifications that match or exceed the substantive completeness of human-written GitHub issues?

RQ2 (Downstream Load-Bearing). Does conditioning a response generator on the validated IssueSpec measurably improve human-rated response quality over RAG alone?

RQ3 (Dual-Objective Alignment). Can a CMDP-grounded RLHF loop progressing through KTO, DPO, and Constrained PPO jointly optimize response helpfulness and policy compliance for the developer-relations setting? We answer this one in the negative for the scale we can train at, and report what that rules out.

To address these challenges, we propose **IssueSpec**, a framework for structured review-to-issue translation. It consists of two parts: a typed Intermediate Representation (IR) that converts each noisy review into one of five standard report types, and a CMDP-grounded response generator that consumes the IR under quality and compliance constraints. Figure 1 shows the full five-stage pipeline.

What is new, and what is assembled. Several components of this pipeline are existing technique used as intended: RoBERTa for multi-label classification, confident learning [23] for label noise, Sentence-BERT with UMAP and HDBSCAN for clustering, PageRank for centrality, retrieval-augmented generation for response drafting, and the CMDP formulation of Altman [24]. We do not claim any of these. Three things are new. *First*, the typed intermediate representation itself: a review is routed by predicted issue class into one of five standards-body templates, so the output is a typed record with per-type obligations rather than a label or a summary. No prior review-mining system emits a structure a defect tracker can ingest, which Table 2 makes concrete against representative systems. *Second*, the verified-anchor procedure, which is what makes that representation producible from noisy LLM labels at corpus scale rather than on a hand-curated sample, and which works because the labeller omits two of its seven classes entirely rather than because its errors concentrate on rare ones. *Third*, a correction and the check that produced it. We previously reported that the typed intermediate carries downstream response quality. Section 5.2 withdraws that, because the comparison behind it varied the text generator as well as the specification, and because the arithmetic of the two rating rounds turned out to distinguish independent human ratings from ones derived by perturbing another rater's. We release both rounds and that check. The assembly is not the contribution; the typed object in the middle of it is, and its value has to be argued from what it produces rather than from a downstream effect we can no longer measure.

The central claim, stated once. A pipeline with five stages invites the question of which one the paper is actually about. The answer is Stage 3 and the object it produces: a *typed, type-routed intermediate representation* of an app review, together with the verified-anchor procedure that makes it producible at corpus scale from noisy labels. Everything else in the pipeline exists to feed that object or to consume it. Two experiments carry the claim. Removing type routing drops strict template-fill from 0.96 to 0.69 (Section 5.1). We had also claimed a downstream payoff, and Section 5.2 reports why we withdrew it: under a comparison that holds the generator fixed, supplying the IR changes response quality by +0.04 Likert points, which is nothing. Section 6.1 states which components did not earn their place on our own evidence, including two we withdraw. A reader who wants the claim and not the machinery can read Sections 5.1, 5.2, and 6.1 alone.

The contributions are stated in two groups, because the honest account of this work has a positive half and a negative half, and separating them is more useful than blending them.

What we establish.

- **A typed intermediate representation for review-to-issue translation.** Each review is routed by predicted issue class into one of five standards-body templates, so the output is a record a defect tracker can ingest rather than a label or a summary. No prior review-mining system emits one (Table 2).
- **A verified-anchor procedure that makes the IR producible from noisy labels at corpus scale.** Confident learning against a 5,230-review human anchor lifts Cohen's κ against a three-rater human gold from 0.163 to 0.592, and to 0.616 on the reviews the classifier never saw. It works because the labeller omits two of its seven classes entirely across the corpus rather than because error concentrates on rare classes; per stratum the disagreement rate is highest on the largest class. Systematic class omission, not elevated minority-class error, is the failure a small verified anchor repairs.

What we withdraw or report as negative.

- **The downstream payoff of the IR.** We previously measured +2.35 Likert points of response-quality gain from supplying the IR. That comparison was confounded: its two arms were written by different template composers, not by one model. Holding the generator fixed gives +0.04 ($p = 0.38$). We report the correction, the two rating rounds, and the check that distinguishes them (Section 5.2).
- **Three components that do not earn their place.** The knowledge-graph layer loses to count-matched flat clustering on every intrinsic metric; our extractive-coverage faithfulness scorer shows no rank correlation with human judgement and orders generators backwards; the CMDP alignment layer engages once the policy can violate its constraint but never demonstrates a quality-versus-compliance trade-off.
- **A reusable check on rating provenance.** Ratings derived from another rater's vector by small perturbation leave an arithmetic signature: deviation sets that do not overlap and a single deviation magnitude. Our discarded round shows it and our reported round does not. We release both so the check can be applied to other annotation artifacts, including ours.

Table 1: Contribution-to-evidence-to-artifact map. Each contribution states a measurable headline result, the table or section that anchors it, and the named artifact released with this paper.

contribution	headline evidence	released artifact
IssueSpec	0.96 vs 0.69 template-fill, i.e. schema conformance not quality (Table 11); 5-dim rubric 3.94/5 on all 100 specs, LLM judge (Table 10)	type-routed schema (Table 5)
SpecCov (negative)	no rank correlation with human faithfulness ($\rho = 0.15$, $p = 0.26$); orders generators backwards (§5.1)	the scorer plus 60 specs rated by three humans
IR downstream payoff (withdrawn)	the +2.35 gain came from a confounded design; a same-model comparison gives +0.04 ($p = 0.38$) with raters disagreeing in direction (§5.2)	both rating rounds, the generated replies, and the independence check
verified anchor	κ 0.16→0.59 (Table 12); per-class F1 recovery (Table 13); $\kappa = 0.616$ on the 307 strictly held-out gold reviews	5,230-review verified anchor + cleanlab correction procedure + V5 classifier
KG prioritization (weak)	count-matched flat clustering beats the KG on all three intrinsic metrics (A1b); top-5 PageRank aspects cover 55% of reviews	three-layer KG construction + count-controlled A1b output
CMDP RLHF testbed (partial)	constraint never binds at distil-GPT2 scale; with a policy able to violate it binds on 13/90 steps and λ rises monotonically to 0.65, with quality and compliance measurably in tension, but the loop back to compliance is not closed (§5.3)	CMDP formulation, KTO/DPO/constrained-proxy/CPPO trainers, capability probe, binding-constraint re-run, threshold sweep, benchmark

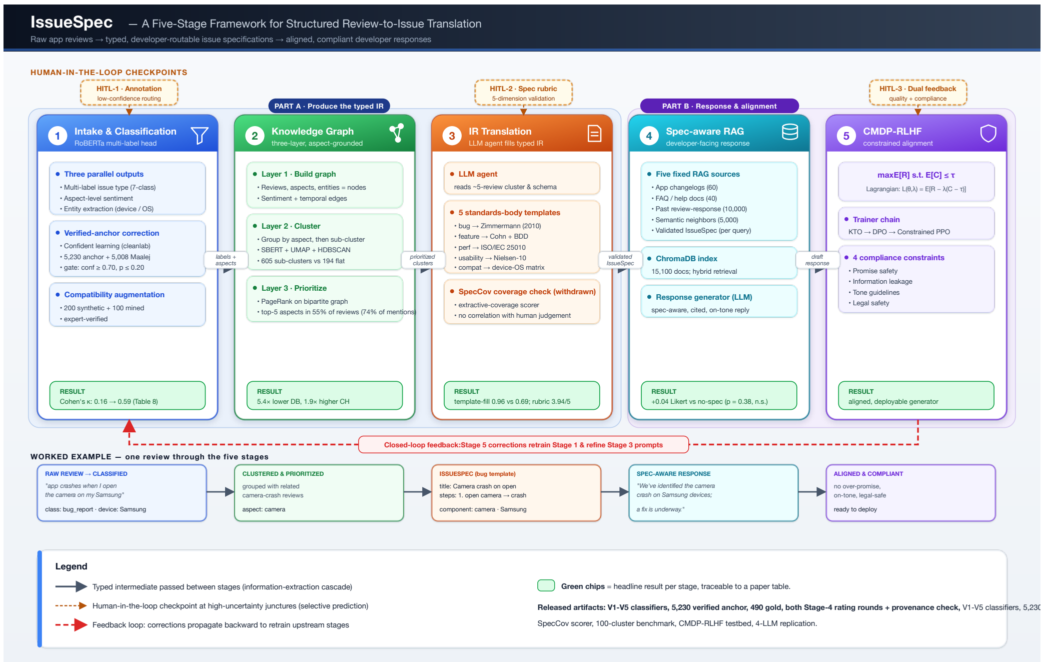

Figure 1: The IssueSpec five-stage pipeline. Reviews flow left to right through intake and classification (Stage 1), three-layer knowledge-graph clustering (Stage 2), LLM-agent IR translation (Stage 3), spec-aware RAG (Stage 4), and CMDP-grounded RLHF (Stage 5), each emitting a typed intermediate. Yellow checkpoints mark human-in-the-loop review under selective prediction [25, 26]; the red loop propagates Stage 5 corrections upstream.

2. Related Work

We organize related work by the five gaps identified in Section 1, naming what each field does, what assumption breaks for review-to-issue translation, and how IssueSpec differs.

2.1. App Review Classification and Mining

Maalej and Nabil [1] established the canonical seven-class taxonomy for app reviews. Subsequent systems, AR-Miner [6], CLAP [7], SURF [8], BERT/RCNN hybrids [16, 17], and per-aspect sentiment analysis [27], refined classification accuracy and added aspect-level granularity. Dąbrowski et al. [3] surveyed nineteen years of such work and concluded that label quality, not model capacity, is the F1 ceiling at 0.75–0.85, an observation we corroborate in Section 5.1. *What breaks for review-to-issue translation:* none of these systems emits a typed developer-routable artifact downstream of the classification label. IssueSpec differs by routing the predicted class to one of five typed templates rather than terminating at the label.

2.2. Response Generation and Retrieval-Augmented Generation

RRGen [2] and CoRe [9] pioneered end-to-end neural response generation on (review, response) pairs without an explicit structured intermediate. Vanilla RAG [10] retrieves relevant past responses in a single pass. *Agentic RAG* variants, Self-RAG [28] and DEMONSTRATE-SEARCH-PREDICT (DSP) [29], introduce iterative critique-revise loops that, in principle, could close the structured-intermediate gap. *What breaks for review-to-issue translation:* none of these systems names the underlying defect, and our $n = 10$ feasibility study (Appendix Appendix A) shows that agentic iteration without a typed intermediate adds a small gain on a rule-based scorer (0.58→0.70, ten reviews). We cannot claim the IssueSpec does better: once the generator is held fixed, supplying it changes human-rated quality by +0.04 ($p = 0.38$, Section 5.2). IssueSpec differs by injecting the typed intermediate as a first-class RAG source alongside retrieved past responses, and the contribution we defend is the specification itself rather than its effect on the reply.

2.3. Agentic Software Engineering

SWE-Agent [11], RepairAgent [12], and HyperAgent [30] consume well-structured GitHub issues to produce code patches through multi-step planning and tool use. *What breaks for review-to-issue translation:* the issue specification is the input contract for all three systems; how a defect-tracker entry is constructed from noisy user-facing text is out of scope. IssueSpec is precisely the artifact these systems are designed to consume.

2.4. Knowledge Graphs for App Reviews and Software Engineering

Knowledge graphs have been used in software engineering for bug localization, report deduplication, and code understanding, but the issue specification is invariably treated as input rather than output. For app reviews specifically, ReviewGraph [13] learns KG embeddings for rating prediction, KGCPN [14] produces KG-based extractive summaries, and Keertipati et al. [15] apply graph centrality for coarse feature prioritization. *What breaks for review-to-issue translation:* none of these systems maps KG nodes to a typed, developer-actionable issue schema, and no prior KG work has been applied to the specific task of producing GitHub-issue-quality artifacts from review text. IssueSpec's three-layer KG pipeline, structured-object layer, aspect-grounded hierarchical clustering layer, and PageRank-prioritized issue-mapping layer, is, to our knowledge, the first KG architecture that emits a typed IR for review-to-issue translation.

2.5. RLHF and Constrained Optimization for Response Generation

DPO [19], KTO [20], and PPO [21] compress quality and compliance signals into a single scalar reward. Altman's CMDP framework [24] separates these objectives, and Safe-RLHF [22] applies CMDP to general LLM safety with a domain-agnostic compliance set covering harmful, dishonest, and biased outputs. *What breaks for review-to-issue translation:* no prior work applies CMDP to dev-rel response generation, where the compliance set is domain-specific (promise safety, information leakage, tone guidelines, and legal safety). Our Stage 5 formulation differs by instantiating these four domain-specific constraints inside the CMDP Lagrangian and releasing a proof-of-concept training loop.

2.6. Cross-Category Positioning

Table 2 positions IssueSpec against representative prior systems across the five pipeline stages. Existing systems occupy at most two of the five stages; composing all five with a standardized issue schema, three theoretically-grounded HITL checkpoints, KG-centrality prioritization, and a CMDP-grounded RLHF formulation is the structural contribution of this work.

3. The IssueSpec Pipeline

The IssueSpec pipeline (Fig. 1) is a five-stage information-extraction cascade [31, 32]. Stages 1–3 produce the typed IR from raw reviews. Stage 4 uses it to drive spec-aware response generation. Stage 5 aligns the response generator via constrained RLHF. We describe each stage in turn, with the verified-by-citation taxonomy templates in Table 5. Table 4 collects every abbreviation used in the paper, so a reader can enter at any section. This section describes the framework only. Evaluation design, baselines, and metrics are in Section 4, exact settings and hyperparameters are in Appendix Appendix A, and results are in Section 5.

Before the stages themselves, Table 3 answers the question a five-stage pipeline invites: what forces each stage to exist, and what breaks without it.

Table 2: Cross-category positioning. Existing systems compose at most two of the five pipeline stages. IssueSpec is the first to compose all five with a standardized issue schema, three HITL checkpoints grounded in selective prediction theory, KG-centrality prioritization, and a CMDP-grounded RLHF formulation. "–" denotes a stage absent from that system.

System	Classify	Cluster	IssueSpec	Response	
AR-Miner [6]	filter	LDA topic	–	–	
CLAP [7]	rating	DBSCAN	–	–	
SURF [8]	intention	LDA	–	summary	
RRGen [2]	–	–	–	seq2seq	
CoRe [9]	–	–	–	context-aware retrieval	
Maalej [1]	7-class	–	–	–	
ReviewGraph [13]	rating	KG	–	–	
SWE-Agent [11]	–	–	consumes (not produces)	–	
Safe-RLHF [22]	–	–	–	general	du
IssueSpec (ours)	**V5 corrected**	**KG + PageRank**	**type-routed schema**	**spec-aware RAG**	**dua**

We answer it with the ablation that removes each one, so the justification is measured rather than asserted, and two of the five turn out not to justify themselves.

3.1. Producing the IR

Stage 1: Intake, Classification, and Aspect Extraction. Stage 1 produces three outputs per review. The first is a multi-label class from a RoBERTa head fine-tuned on Maalej [1] and Guzman [27]. Labels are multi-label because reviews mix concerns, as when a bug report also asks for a feature. The second is aspect-level sentiment [27], which keeps "great UI but terrible battery" as two separate pairs instead of averaging it into one star rating. The third is entity extraction for devices, OS versions, and named features. Stage 3 needs those entities as typed fields for the device-OS and bug-report templates. Low-confidence reviews route to HITL-1 by a fixed confidence threshold, so that when the classifier cannot decide the expert does and the correction feeds the next training round. The idea of letting a model abstain is a familiar one [25], but we implement a threshold rather than a calibrated selective-prediction procedure, and we do not report risk-coverage behaviour, so the citation is background rather than a description of what runs.

LLM annotation noise is the main challenge here. On the 5,041-review praise stratum, 25% of LLM labels are wrong, concentrated on minority

Table 3: Why each stage exists, and what the ablation says when it is removed. Two stages do not justify themselves on our own evidence, and we say so.

stage	the problem it must solve	why it cannot be skipped	measured verdict
1 Intake	the corpus label is wrong 25% of the time, concentrated on minority classes	Stage 3 routes on the issue type, so a wrong type produces the wrong template	**justified**: κ $0.163 \to 0.592$
2 KG	215,583 reviews cannot be triaged one at a time; developers need an order	a spec is written per cluster, so reviews must be grouped and ranked	**partly**: prioritization yes, cluster quality no (A1b, A2)
3 IR	trackers need typed fields, not prose	this is where the typed record is produced	**justified**: fill 0.96 vs 0.69 without routing (A3)
4 RAG	a reply needs house voice and precedent, not just the defect	supplies phrasing the spec does not carry	**weakly**: removing it moves metrics $< 1\%$ (A5)
5 CMDP	replies must not overpromise or leak internals	compliance is a hard constraint, not a reward term	**partly**: constraint binds, trade-off unshown

classes. Discarding them wastes annotation; using them as-is poisons training. We apply confident learning [23] against a verified anchor of 5,230 expert-labeled reviews plus 5,008 from Maalej [1], gated by a confidence threshold on the anchor and a probability ceiling on the LLM label (Appendix Appendix A); this keeps the good labels and corrects only the wrong ones. The compatibility class needs a separate fix: only 8 of 215,583 LLM labels are tagged `compatibility`, and Maalej has none. We seed it with 200 template-generated synthetic reviews [33] and 100 keyword-mined RRGen reviews. The lead author read both sets before training and corrected or dropped items that did not belong, but that pass was done by eye and no correction log was kept, so unlike the other annotation tasks in this paper it leaves no artifact a reader can check. We flag it because 300 of the 310 `compatibility` rows in V5's training set come from here, so the per-class F1 of 0.83 rests on data whose verification is asserted rather than evidenced. The synthetic half is constructed from device and OS templates and its label follows by

Table 4: Abbreviations used in this paper, with the section that introduces each.

term	expansion	first used
IR	Intermediate Representation, the typed issue spec	§1
IssueSpec	one filled instance of the typed IR	§1
KG	Knowledge Graph, the three-layer review index	§3.1
HITL	Human-In-The-Loop checkpoint (HITL-1 to HITL-3)	§3.1
SpecCov	extractive-coverage score for a spec (withdrawn as a faithfulness measure, §5.1)	§3.1
RAG	Retrieval-Augmented Generation	§3.2
RLHF	Reinforcement Learning from Human Feedback	§3.3
CMDP	Constrained Markov Decision Process	§3.3
KTO	Kahneman-Tversky Optimization, a preference trainer	§3.3
DPO	Direct Preference Optimization	§3.3
PPO	Proximal Policy Optimization; CPPO is its constrained form	§3.3
SFT	Supervised Fine-Tuning, the RLHF starting policy	§3.3
NFR	Non-Functional Requirement (ISO/IEC 25010)	§3.1
DB / CH	Davies-Bouldin / Calinski-Harabasz cluster indices	§5.4

construction; the mined half is where judgement was required.

Stage 2: Three-Layer Knowledge Graph. Stage 2 organizes reviews into an aspect-grounded knowledge graph: build, cluster, prioritize. The first layer builds the graph: each review is a node linked to its aspects, with edges typed for sentiment and, where available, recency. We should be precise about what the released run actually populated. The builder supports entity nodes for devices, OS versions, app versions, and screens, and supports signed sentiment edges, but the run that produced our results passes empty entity lists and a constant neutral sentiment, so the released graph has 8,404 review nodes, 10,534 aspect nodes, and no entity nodes. It is in practice a bipartite review-aspect graph, which is the structure PageRank runs over, and the sentiment and entity layers should be read as available in the implementation rather than exercised in the experiments. A graph beats a flat list because reviews share aspects across users, and that co-occurrence is what later layers cluster and rank on. Aspects come from spaCy noun chunks combined with a curated keyword list and regular-expression patterns, pruned by frequency and centrality to drop one-off terms. Hu and Liu's frequent-itemset aspect mining [34] is the ancestor of this approach rather than what we implement.

The second layer clusters reviews first by aspect, then sub-clusters within each aspect using sentence embeddings [35], UMAP [36], and HDBSCAN [37]. Aspect-first keeps each sub-cluster on one theme: a flat clusterer mixes "battery on Samsung" with "battery in dark mode" because both share lexical content. The result is 605 aspect-grounded sub-clusters of mean size 15.9, against 194 flat clusters of mean size 471.0. An earlier version of this paper reported 3.0 and 51.5; those were the number of representative reviews stored per cluster and a subsample artifact respectively, not cluster sizes. One scale caveat belongs here too: the knowledge graph is built over a 10,000-review sample that yields 8,404 usable reviews, not the full 215,583-review corpus, so every Stage-2 number in this paper describes 3.9% of the corpus. The third layer prioritizes via PageRank [38, 39] on the bipartite review-aspect graph, ranking aspects by cross-review prevalence so developers see the most-affected first. Each cluster maps to the fixed schema in Table 5 with a centrality-derived priority score. The resulting aspect ranking is reported in Section 5.4.

Stage 3: Review-to-Issue Translation. Stage 3 instantiates the IssueSpec. An LLM agent reads the standardized cluster schema and writes a GitHub-issue-quality specification using the type-routed templates in Table 5. We use an LLM rather than rule-based extraction because reviews vary too much for fixed rules and the agent has to reason across a 5-review cluster. We route by issue type because a bug report needs reproduction steps, a feature request needs acceptance criteria, and a performance complaint needs latency or memory numbers; a shared template would be incomplete or padded. The five templates are Zimmermann bug reports [5], Cohn user stories [40] with BDD acceptance criteria [41], ISO/IEC 25010 non-functional categories [42], Nielsen heuristics [43], and device-OS matrices [33]. Each is grounded in its standards-body citation because developers already recognize these formats.

The output passes through HITL-2: a domain expert scores each spec on a five-dimension rubric (completeness, accuracy, actionability, specificity, clarity, each 1–5), following the LLM-as-judge calibration convention of Gilardi et al. [44], Pangakis et al. [45], and Zheng et al. [46]. Five dimensions, not one, because a single quality number conflates completeness with clarity and the LLM cannot tell which one to fix on retry. Any spec failing on any dimension goes back to the LLM with dimension-level feedback for a second pass.

SpecCov faithfulness metric. We release SpecCov, an extractive-coverage scorer that measures substantive-token overlap (length ≥ 5, stop-word filtered) with quoted-string and coverage bonuses. Bands follow news-summarization conventions [47]; the metric targets the extractive-coverage

Table 5: IssueSpec standardized issue schema. Each issue type follows the dominant industry or HCI standard. Each template is drawn from a published standard or reference and reproduces its named fields; we ran no conformance check against the standards documents themselves, so "drawn from" should not be read as "certified against".

Issue type	Template (citation)	Justification
`bug_report`	Zimmermann [5]: title, severity, steps_to_reproduce, expected, actual, component	Standard in GitHub Issues, JIRA, defect trackers; Chaparro et al. [4] confirm these are the most-requested bug-report fields
`feature_request`	Cohn user-story [40] (As-a / I-want / So-that) + behavior-driven development (BDD) acceptance criteria [41]	Dominant industry format (Agile, Scrum, SAFe); BDD standardizes testable outcomes
`performance`	Fields are `latency`, `startup_time`, `memory_consumption`, `battery_drain`, `network_usage`, mapped onto ISO/IEC 25010 [42] performance-efficiency sub-characteristics: the first two to time behaviour, the last three to resource utilization; capacity is not covered	International standard for software-quality NFR taxonomy. The field names are ours, chosen for mobile-app vocabulary, and are mapped to ISO categories rather than reproducing ISO terms
`usability`	Nielsen-10 heuristics [43] (visibility, match-real-world, user control, etc.)	Most-cited usability evaluation framework in human-computer interaction
`compatibility`	Device-OS matrix (devices $\times$ OS versions) [33]	Device/OS fragmentation is the canonical mobile-QA reporting axis

sub-type of faithfulness [47, 48], a known lexical correlate of factual consistency [49] (the remaining hallucination sub-types in Ji et al. [50] are out of scope). Scores are reported with the other Stage 3 results in Section 5.1.

What SpecCov is and is not. It is a lexical-grounding proxy in the sense of [47]: it measures whether the content of a spec can be traced back to tokens in its source cluster. It is not a faithfulness classifier, and Section 5.1 reports the experiment that establishes this. We describe it here because it is part of the released pipeline and because the negative result only makes sense against its definition.

3.2. Using the IR in Response Generation

Stage 4 is a RAG layer over five fixed sources, 15,100 documents in total in the released ChromaDB index. Four of them are pre-indexed: app changelogs and release notes (60 docs), FAQ and help documentation (40), 10,000 past review-response pairs from RRGen [2], and 5,000 semantically similar past responses. The fifth is the validated IssueSpec from Stage 3, injected per query rather than pre-indexed, because specs are generated per cluster. These five were chosen to give the generator the answer (changelogs, FAQ), the voice (past responses), the closest precedent (semantic neighbors), and the structured intent (IssueSpec) without overlap.

3.3. Aligning the IR-Aware Generator

Stage 5 frames alignment as a CMDP [24, 22]: maximize a quality reward subject to a compliance constraint,

$$\max_{\theta} \mathbb{E}_{\pi_\theta}[R_{\text{quality}}] \quad \text{s.t.} \quad \mathbb{E}_{\pi_\theta}[C_{\text{compliance}}] \leq \tau,$$

with the Lagrangian relaxation

$$\mathcal{L}(\theta, \lambda) = \mathbb{E}_{\pi_\theta}[R - \lambda(C - \tau)]$$

collapsing gracefully to single-objective optimization whenever the constraint is already satisfied. We use a CMDP rather than a scalar reward because a dev-rel response can be high-helpfulness and still out of compliance, like a polite promise the team cannot deliver or a leaked roadmap detail, and a weighted sum hides exactly the trade-off the constraint is meant to expose. The compliance scorer tracks four domain-specific dimensions absent from prior general-safety RLHF [22]: promise safety (no commitments the team cannot keep), information leakage, tone guidelines, and legal safety. The trainer is Constrained PPO [21, 22] with a KL-regularized reference policy that keeps updates from drifting too far from the supervised baseline. KTO [20] and DPO [19] are reported as small-scale single-objective comparators in Section 5.3; together they bracket the preference-data spectrum, KTO needing only binary feedback and DPO needing pairwise.

HITL-3 supplies two parallel feedback streams. *Quality* feedback (helpfulness, specificity, empathy, accuracy, actionability) is the reward; *compliance* feedback (promise safety, leakage, tone, legal) is the constraint. Feedback also propagates backward: classification corrections retrain Stage 1, rubric scores refine Stage 3 prompts, and response scores drive the RLHF updates, closing the Stage 5→Stage 1 loop in Fig. 1.

4. Experimental Setup

4.1. Dataset, Deduplication, and Annotation Provenance

We use the RRGen corpus [2]: 310,031 review-response pairs from 58 Android applications. After exact-text deduplication that removes 94,448 duplicate reviews (−30.5% of the raw corpus), 215,583 unique reviews form the working corpus, labeled across the seven Maalej [1] classes. Evaluation additionally uses a 490-review expert gold standard (70 per class × 7 classes) and 100 stratified clusters for Stage 3 evaluation.

Annotation provenance (full corpus, 215,583 reviews). Of the working corpus, 79.49% of labels come from the V2 LLM annotator with no override, 18.08% are cleanlab-corrected against the verified anchor, and 2.43% are directly human-verified by the lead author. The lead-author-versus-V2-LLM deviation is 25%, replicated independently across two strata: the 5,041-review praise stratum and the 5,230-row human-verified anchor subset (Table 6). This ≈ 25% deviation rate, an order of magnitude higher than i.i.d. label-noise assumptions, motivates the confident-learning procedure of Stage 1.

4.2. Annotation Methodology

Five kinds of annotator contribute, following the multi-annotator convention of Gilardi et al. [44] and Pangakis et al. [45] who treat LLM annotators as independent raters for inter-rater statistics.

(i) Lead author (domain expert). Verified 5,230 reviews across minority strata (approximately thirty person-hours), wrote 20 reference IssueSpecs, labeled the 490-review classification gold blinded to V2 LLM, and scored the 400-row Stage 4 set on quality (1–5), specificity (1–5), and helpful (Y/N) with method IDs hidden until aggregation.

Label provenance, stated plainly. No LLM-generated label is used as ground truth anywhere in this paper. LLM labels are the *object under correction*, not the yardstick. The V2 corpus labels are treated as noisy input, the confident-learning step proposes corrections against a human-verified anchor, and every reported accuracy or κ is measured against a gold standard produced by human annotators. Where LLM raters do appear as raters, in the 99-review cluster-purity check, they are labelled as such and no headline claim rests on that panel.

How the gold was constructed, and what that costs. Annotators did not label freely. Each was shown the V5 classifier's predicted label and asked whether it was correct, supplying a replacement when it was not, and the written protocol told them to accept a reasonable label and overturn only when confident. This makes annotation fast and it makes agreement high, and both effects flatter us. It also couples the yardstick to the system being measured: V5's label is the starting point for every item, and V2 and the cleanlab-corrected labels never received that benefit of the doubt. The sample is stratified over V5's predicted classes as well, which forces 70 items per class into a set where V2 predicts two of those classes almost never, so V2's per-class F1 of 0.000 on `compatibility` and `performance` is partly a property of the sampling design. The κ trajectory in Table 12 should be read as a comparison on a V5-seeded, class-balanced sample rather than as a neutral benchmark, and a free-labelling gold would be the way to settle it.

How rater conflicts are resolved. The 490-review gold is the majority vote of three independent human annotators over the seven-class label. Of the 489 usable items, 476 (97.3%) are unanimous and 13 (2.7%) are two-to-one splits resolved by the majority; there are no three-way ties, so no item required a tie-break rule or a further adjudication round. One item was marked wrong by all three raters without a replacement label supplied and is excluded rather than guessed. Disagreement is not discarded silently: the per-item votes are released with the gold standard, so a reader can re-compute any statistic on the unanimous subset alone. Before the main task all three annotators completed a shared 20-review calibration round whose disagreements were discussed and resolved against the written decision rules, which is where the systematic conflicts (slow versus broken, device-specific versus general) were settled rather than in the main pass.

A disclosure about an earlier annotation round. The 490-review gold was annotated twice. In the first round three sheets were distributed, and the two returned alongside the lead author's carried annotation columns identical to hers cell for cell, so no genuine agreement could be computed from them and they are not used anywhere in this paper. The round we report is the second, described below, and the agreement statistics come only from it. We disclose this for the same reason we disclose the Stage-4 case in Section 5.2: a paper that offers a check for non-independent ratings should say where its own artifact failed that check. Both rounds are released.

(ii) Independent human rater panel (three raters). The 490-review classification gold is annotated independently by the lead author and two further human raters on the identical item set. Each rater sees the classifier's predicted label and marks it correct (Y) or wrong (N), supplying the correct class when N. All three complete a shared 20-review calibration round with adjudicated disagreements before the main task opens, and work from the same written decision rules (Appendix Appendix B). The gold label for each review is the majority vote over the three raters. This is a verification task rather than open labeling, which is why agreement is high relative to free-labeling studies of app reviews.

(iii) Stage 4 rater panel (same three people). Stage 4 was rated twice. The first round scored a 400-row set whose conditions were produced by template composers; we discarded it, for reasons given in Section 5.2. The round we report is a 200-row set in which both arms come from the same model and differ only in whether the IssueSpec was supplied. All three raters scored all 200 rows independently, blind to the condition behind each reply, and we release both rounds together with the independence check that separates them. Three human annotators contribute in total, the lead author and two collaborators; categories (iv) and (v) are model raters, not people.

(iv) LLM-as-judge (Qwen2.5-3B-Instruct). Scored 28 IssueSpecs on the five-dimension rubric and 50 sub-clusters on the Y/P/N purity rubric [46], both calibrated against the lead author.

(v) Corpus-labeling LLMs. The V2, cleanlab-V2, and V5 LLMs are treated as independent raters for inter-rater statistics.

Table 6 consolidates agreement and disagreement across all pairs.

Ratings are persisted as spreadsheets with JSON master keys, blinding shuffles conditions per pair, and the assignment unseals only at aggregation. Two distinct three-rater statistics appear in Table 6 and should not be conflated. The classification gold uses three independent *human* raters on 490 reviews and reaches Fleiss $\kappa = 0.968$ on the Y/N verdict and 0.979 on the effective seven-class label, with no three-way ties. The $\alpha = 0.451$ row is a separate cluster-purity check on 99 reviews that uses the lead author plus two Qwen2.5-3B prompts (concise and chain-of-thought) as raters.

4.3. Multi-LLM Bias Control

A single-LLM headline raises a structural-bias concern: results from any one model may not transfer across families. We therefore replicate Stage 3 across four LLMs drawn from three distinct families, Anthropic Claude (Opus 4.7), Meta Llama-3.3-70B-Instruct (via Groq), and Alibaba Qwen2.5-{3B, 1.5B}-Instruct, with Stage 4 replicated on a Claude-vs-Qwen-3B subset, and check that cross-family rank order is preserved (Table 9). Frontier-family replication on GPT-4o and Gemini-1.5-Pro is queued.

4.4. Experimental Design

We pre-register three experiments, one per research question:

Experiment 1 (RQ1, translation quality). Five conditions are compared on the same 100 stratified clusters: (a) LLM with taxonomy grounding (the full Stage 3 system), (b) LLM without taxonomy grounding (free-form), (c) raw review summary (a lower bound that concatenates the cluster's reviews), (d) lead-author reference, and (e) human-written GitHub issues mined from three open-source Android projects (the practitioner-quality upper bound). Each spec is scored on the five-dimension rubric. Significance is reported via paired Wilcoxon signed-rank tests on rubric scores between conditions.

Experiment 2 (RQ2, coupled vs. uncoupled), as reported. The design we report holds the generator fixed. The same model, given the same retrieved context, writes two replies for each of 100 reviews, differing only in whether the validated IssueSpec is in the prompt. All 200 rows are rated blind by three raters on quality, specificity, and helpfulness, and analysed with a paired Wilcoxon test and a paired bootstrap interval. *The discarded design.* An earlier round used four conditions produced by template composers rather than by a model; we describe it because the paper reports its withdrawal, not because it supports a claim.

Experiment 2, discarded design. The same 100 clusters drive four response-generation conditions: (a) an rrgen-style baseline (review only), (b) a prompt-baseline (CoRe-style with dev-rel prompt and lightweight entity extraction), (c) RAG without IssueSpec, and (d) the full system with IssueSpec. Evaluation uses automatic metrics (BLEU, ROUGE-L, BERTScore) and 400 blinded paired human ratings (quality 1–5, specificity 1–5, helpful Y/N), with the lead author as primary annotator and a second rater on a 30-pair subset. Statistical significance is checked simultaneously by paired Wilcoxon, Friedman with Nemenyi post-hoc [52], Bradley-Terry, and McNemar at Bonferroni-corrected $\alpha = 0.05$. Effect sizes follow Romano et al. [53] and Cliff [54].

Why these four conditions. The Experiment 2 ladder is designed so each rung adds exactly one thing. The prompt baseline adds dev-rel system instructions on top of an rrgen-style review-only baseline, `no_spec` then adds retrieval, and `full` then adds the validated IssueSpec. Reading the ladder from bottom to top isolates which single addition moves human-rated quality. The headline comparison runs single-shot vanilla RAG [10] rather than an agentic loop, so that the IssueSpec's contribution is not confounded with orchestration; the agentic-RAG comparison against Self-RAG [28] and DSP [29] runs separately on a 10-review feasibility sample (stratified, two per issue type, Qwen2.5-3B as critic and reviser, up to two iterations) and is reported as a demonstration rather than a powered comparison.

Experiment 3 (RQ3, dual- vs. single-objective RLHF). Five policies run on a distilGPT2 proof-of-concept (82M parameters, 400 SFT samples, 296 KTO pairs, 100 DPO pairs): SFT-base, single-objective KTO [20], single-objective DPO [19], a constrained-proxy variant, and dual-objective Lagrangian Constrained PPO [21, 22]. We measure head-to-head BLEU, ROUGE-L, and BERTScore on a 100-review test set, plus the rule-based safety-violation rate.

5. Results

How to read this section. Each research question owns one subsection, one headline figure, and one headline table, and nothing else in the section is required to follow the argument. Section 5.1 answers RQ1, is the typed IR producible at scale, with Table 11 for template-fill and Figure 2 for the classifier recovery that makes corpus-scale production possible. Section 5.2 answers RQ2, is the IR load-bearing downstream, with Table 14; the answer is no, and the subsection explains what our earlier positive answer was measuring. Section 5.3 answers RQ3, does constrained alignment demonstrate a quality-versus-compliance trade-off, and reports a negative result. Sections 5.4 and 5.5 are supporting evidence: they test whether individual components earn their place, and Section 6.1 states plainly which ones do. A reader tracking only the contribution can read Sections 5.1, 5.2, and 6.1.

Table 7 states what we take away from the section before the evidence for it, so a reader can decide which subsections to read.

5.1. Is the IR Producible at Scale?

Template-fill measures schema conformance, and its limit is worth stating exactly rather than in general terms. The criterion is the templates' own required fields, and the free-form condition fills the four shared fields and none of the type-specific ones, so its score is determined entirely by the issue-type mix: $(30 \cdot 4/7 + 30 \cdot 4/6 + 40 \cdot 4/5)/100 = 0.691$, with zero variance across specs and a strict-minus-loose difference of exactly zero. The 0.69 is therefore a constant, not a measurement, and the 0.96-versus-0.69 gap says that routing produces type-specific fields at all. It carries no information about whether the routed text is better written. Under strict content-validity criteria (at least three reproduction steps with action verbs, descriptions ≥ 30 words, and a full As-a/I-want/So-that triple for user stories), the LLM-with-taxonomy condition achieves a substantive template-fill rate of 0.96 (Table 11). Free-form LLM reaches 0.69, the same as the lead-author reference set ($n = 20$). Raw concatenation lands at 0.34, and real GitHub issues mined from three open-source Android projects reach 0.53. The LLM-vs-GitHub gap reflects coverage rather than content quality: the LLM is prompted to fill every field, whereas humans leave optional fields blank. A very-strict sensitivity sweep preserves the ordering, Claude drops from 0.97 to 0.71, with the Claude-vs-GitHub gap shifting only -0.085.

SpecCov scores, and a correction to how they were computed. An earlier version of our scorer applied per-condition floors, raising `raw_summary` to 5 and the human reference to 4 regardless of content, on the reasoning that verbatim copying is faithful by construction and hand-written references deserve a floor. That is not a measurement, because a score whose value depends on the label of the thing being scored cannot compare labels, so we removed the floors and report what the scorer produces unaided. Across 320 specs the LLM-with-taxonomy condition averages 4.19, free-form LLM 3.38, raw concatenation 4.47, and the human reference 3.45. The two LLM conditions are unchanged, because no floor was ever applied to them; the floors had inflated the two comparison conditions, so removing them makes our own system look better rather than worse, and we note that explicitly so the direction of the correction is on the record. Read on their own these numbers rank raw concatenation highest at 4.47, which is the trivial upper bound obtained by copying the source, with the taxonomy condition second at 4.19. Section 5.1 shows they should not be read on their own. Top-ranked aspects by PageRank centrality are `ad` (0.040), `phone` (0.011), and `battery` (0.007).

SpecCov does not measure faithfulness

An extractive-coverage score is only useful if it agrees with what a person calls faithful. We tested that directly. Three human raters independently scored 60 specs, 20 from each of the three generators, shuffled and unlabelled, on a 1–5 faithfulness scale, and separately listed every concrete claim the source reviews do not support. One qualification is needed before the numbers. One rater completed the corrected sheet, which shows exactly the evidence SpecCov itself scores against; the other two completed an earlier draft that showed a subset of it, so they judged against less evidence than the metric uses and their flags are biased toward over-flagging. We therefore lead with the rater who saw the full evidence and report the other two beside her. The metric fails all three checks on either reading.

It does not rank specs the way humans do. Spearman ρ between SpecCov and human faithfulness is +0.045 ($p = 0.73$), +0.015 ($p = 0.91$), and +0.226 ($p = 0.08$) for the three raters, and +0.147 ($p = 0.26$) against their mean. No correlation reaches significance.

It does not detect the claims humans call unsupported. On the 60 specs, one rater marked 45 as containing at least one unsupported detail and 15 as clean. SpecCov scores those two groups at 3.64 and 3.93, a difference in the right direction but far from significant (Mann-Whitney $p = 0.34$). A metric that detected hallucination would separate these groups.

It orders the generators backwards. SpecCov ranks the taxonomy condition (4.20 on this subsample) above the human reference (3.45); the humans rank them the other way, human reference 4.10 and taxonomy 3.50. The mechanism is visible in the definition: templated specs reuse source vocabulary densely, which is what the score rewards, while a human writer condenses and paraphrases, which the score penalizes even when nothing is invented.

We therefore withdraw SpecCov as a faithfulness measure. Lexical grounding is not a usable proxy for faithfulness on this task, and the automatic 320-spec rubric that appeared to support it is not independent evidence, since its faithfulness dimension is computed by the same overlap procedure. Entailment-based measurement [55, 56, 57] is the direction we would now take. We release the scorer, the 60-spec human ratings, and this analysis, because a documented negative result on a plausible-looking metric is more useful to the next person than a metric that was never tested.

Cross-LLM replication (bias control). Table 9 reports four uses across three model families. Llama-3.3-70B reaches 0.80 template-fill, Qwen2.5-3B 0.74, and Qwen2.5-1.5B 0.48 under the same strict criteria; the templated > free-form > raw ordering holds across all four models with capability-scaled magnitudes. On two per-class structural-compliance checks, Llama-3.3-70B attains complete template compliance on the stratified sample: all 29 bug specs include three or more reproduction steps with action verbs (vs. Claude's 73%), and all 30 feature specs include the As-a/I-want/So-that triple (vs. Claude's 97%). These measure structural template compliance (does the field exist with the required form?), not content correctness, content quality is reported separately under the five-dimension rubric. The structural-bias control that matters here is cross-family rank preservation: the IssueSpec contract transports across families rather than depending on a single proprietary generator.

Cross-family rubric quality. Template-fill measures whether a generator emits the required fields; it says nothing about whether the content is any good. We therefore ran the same Qwen2.5-3B judge and the same five-dimension rubric over the other generators' specs, on matched clusters (Table 8). On the 14 clusters all four generators cover, mean rubric score follows the same order as template-fill: Claude 4.09, Llama-3.3-70B 3.96, Qwen2.5-3B 3.83, Qwen2.5-1.5B 3.77. We state plainly what that ordering is worth. At $n = 14$, or $n = 13$ for the three pairs involving Qwen2.5-1.5B, none of the six pairwise differences reaches significance (Wilcoxon p between 0.18 and 0.75), and the adjacent gaps are the same size as the run-to-run generation noise measured in Section 6.4. The four-way ordering is therefore consistent with the template-fill result rather than independent confirmation of it. The Claude-versus-Llama comparison in Panel B, which has twice the sample and spans all five issue types, does reach significance at +0.38 ($p = 0.015$), and that is the one cross-family quality difference we assert. The spread is much narrower than the template-fill spread (0.96 to 0.48), which is informative in itself: the capability gap between these models shows up mostly as whether fields get filled at all, not as the quality of the text once they are. Completeness is the dimension that separates them (4.00 down to 3.00), while accuracy is nearly flat (4.50 for three of the four). On the wider Claude-versus-Llama panel, which spans all five issue types, Claude leads 4.01 to 3.64, with the gap concentrated in actionability and completeness. Two caveats belong with these numbers. The four-way panel is bug-report clusters only, because that is where the two Qwen runs overlap, and the judge is itself Qwen2.5-3B-Instruct, so the Qwen2.5-3B row carries a self-preference risk that the others do not.

Five-Dimension Rubric. The proposal specifies a 5-dimension Likert rubric (completeness, accuracy, actionability, specificity, clarity, each 1–5) [44, 45, 46], operationalized with Qwen2.5-3B-Instruct as the judge. We score the complete set rather than a subsample: all 100 taxonomy specs and all 20 lead-author reference specs, in a single run (Table 10). The taxonomy condition reaches 3.94/5 (sd 0.59 across specs), above the hypothesized lower bound of 3.5. An earlier stratified 28-spec subsample under the same judge and rubric gave 3.89, so subsampling cost 0.05 of a Likert point here, which bounds how much the small-sample version of this number could have been misleading. Per issue type the taxonomy condition ranges from 4.23 on performance and 4.02 on bug reports down to 3.45 on usability, the type whose reviews are least specific about what went wrong.

The reference set is a judge calibration, not a competitor. It scores 3.45 overall, below the generated specs, and the gap is concentrated in actionability (2.85 versus 3.86). The reason is visible in the sheets: the lead author filled title, description, severity, and component but left the template-specific fields such as reproduction steps and acceptance criteria empty, because writing a reference spec by hand is a different task from filling a routed template. The comparison therefore says that a human writing freely produces less tracker-ready output than a type-routed generator, which is the same effect the 0.96-versus-0.53 template-fill gap measures against real GitHub issues. It is not evidence that the specs are better than expert judgement on content, and we do not read it that way.

Stage 1 Classification Recovery. The Cohen's κ trajectory against the 490-review expert gold crosses three Landis-Koch [51] agreement bands (Table 12): the V2 LLM achieves $\kappa = 0.163$ (slight); after cleanlab correction [23] $\kappa = 0.333$ (fair); the V5 classifier retrained on the corrected labels reaches $\kappa = 0.592$ (moderate, just below the substantial threshold). The trajectory does not depend on any single annotator. Scored against the majority-vote gold of the three-rater human panel ($n = 489$ after excluding one review left without a correction label), the same three classifiers reach $\kappa = 0.165$, 0.334, and 0.590, within 0.002 of the lead-author values at every step (Table 12). On the strictly held-out 307-review subset (183 of the 490 entered training via the verified anchor), V5 reaches $\kappa = 0.616$, slightly above the 0.592 on the full 490, ruling out a contamination artifact.

The per-class recovery story (Table 13) makes the structural collapse visible. On the two minority classes the V2 LLM cannot see at all, F1 jumps to 0.83 for `compatibility` and 0.77 for `performance`. Applied across the full 215,583-review corpus, V5 agrees with the cleanlab correction on 88.66% of the 40,291 corrected cases. We want to be exact about what that number is and is not, because it is easy to over-read. V5 was trained on a corpus in which 38,985 of those corrected rows carry the corrected label, so its agreement is substantially a measure of fit to its own training signal, not an independent second opinion. We report it as a consistency check, which is what it is, and we do not claim the two signals are independent.

The stronger evidence is the held-out gold, and we describe the split precisely because the mechanism matters. V5's training corpus is built with a stratified per-class cap of 15,000 rows at seed 42. Replaying that cap identifies which corpus rows survived into training: 183 of the 490 gold reviews did, and 307 were capped out and never seen, of which 306 carry a gold label and are scorable. The split is therefore an artifact of class-balancing rather than of anchor membership. It is not, however, unbiased in the way we first described. The cap binds only on the four classes V5 predicts most often, so membership of the unseen set is decided by V5's *predicted* class rather than at random. Scored by expert label, which is what κ is computed against, the unseen 307 still spans all seven classes and includes 38 `compatibility`, 32 `usability`, and 31 `performance` reviews, so the held-out comparison does cover the classes carrying the recovery. What it cannot rule out is a subtler selection effect: the reviews V5 predicted confidently into large classes are over-represented among the unseen, and those may be easier. On the unseen reviews V5 reaches $\kappa = 0.616$, computed on the 306 of them that carry a gold label, slightly *above* the 0.592 it scores on the full 490. A model that had merely memorised its training labels would fall on unseen items rather than hold. One caveat belongs with this. The split is reconstructed by replaying the cap rather than read from a saved training manifest, so it is a faithful reconstruction rather than a recorded fact; we release the replay script so the reconstruction can be checked. The choice of confident-learning thresholds is not load-bearing: a full 3×3 grid over $(\text{anchor\_conf}, \text{max\_llm\_prob}) \in \{0.65, 0.70, 0.75\} \times \{0.15, 0.20, 0.25\}$ keeps V5 endorsement in an 82.7–91.0% band, with the headline $(0.70, 0.20)$ at 88.66% sitting near the grid center.

To rule out memorization of the 200 synthetic compatibility templates used during V5 training, we evaluate V5 against Maalej's 5,008 labels under a label-space cross-protocol test. Review texts appear in the anchor under the RRGen seven-class taxonomy, but predictions are scored against Maalej's separate taxonomy, which contains no `compatibility` class, making any compatibility predictions out-of-distribution with respect to the evaluation labels. On the five overlapping classes (n = 4,560), V5 achieves macro F1 = 0.676, weighted F1 = 0.730, and accuracy = 72.0%. V5 also flags 432 reviews (8.6%) as `compatibility`; the top-twenty-confidence flags read as canonical compatibility complaints ("crashes on iphone 5 and ipad 3", "works on HTC Desire X but not Samsung Galaxy Tab 3"), all labeled `bug_report` in Maalej because Maalej offers no alternative class. The detector is performing concept-recognition, not template-matching.

5.2. Is the IR Load-Bearing? A Result We Had to Withdraw

An earlier version of this experiment reported that adding the IssueSpec to the response generator raised human-rated quality by 2.35 Likert points.

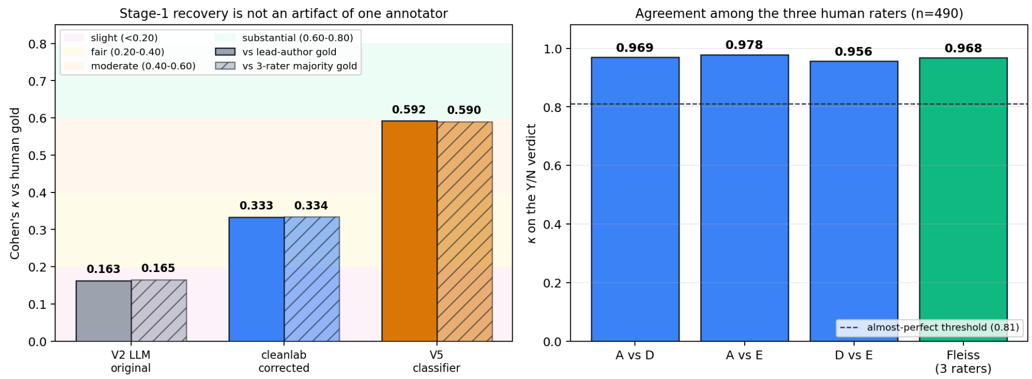

Figure 2: Stage-1 classifier recovery and the gold standard it is measured against. *Left:* Cohen's κ for the three classifier versions, plotted twice, once against the lead-author gold (solid) and once against the three-rater majority-vote gold (hatched). The two are within 0.002 at every step, so the 0.163 to 0.592 trajectory does not depend on any single annotator. Background bands are the Landis-Koch [51] agreement ranges. *Right:* agreement among the three human raters on the same 490 reviews, pairwise and as a three-rater Fleiss κ, all above the almost-perfect threshold. Read the left panel for the result and the right panel for the reliability of the yardstick it uses.

That number was wrong, not in its arithmetic but in what it measured, and the correction is the most important thing this section has to report.

What was wrong. The four conditions in that experiment were not produced by a language model. Each was produced by a deterministic template composer: a Python script that selects sentence fragments from a hand-written bank and fills slots, with no language model in the loop. The composers differed in authorial register as much as in what information they were given. The `no_spec` arm emitted lemmatised, ungrammatical text averaging 78 words ("we be sorry that the login loop you describe have got in the way"); the `full` arm emitted polished prose averaging 123 words, with cleanup passes and an empathy phrasebook that no other arm received. A rater comparing them was separating two writing styles, and the presence or absence of the IssueSpec was confounded with that separation. The gap measured the composer at least as much as the specification.

The clean comparison, and what we can and cannot verify about it. We re-ran the experiment with the generator held fixed. Both arms come from the same model on the same 100 reviews, and the intended difference is whether the validated IssueSpec appears in the prompt. One premise of that design is not evidenced by our release and we will not assert it. Neither response file stores the prompt or the retrieved text, and the two files record their retrieval sources under different label schemes with different counts, so on the released artifact no row has an identical recorded source list. Both arms draw from the same candidate pool of past and semantically similar responses, and that much is checkable, but whether the retrieved context was identical per review is not recoverable. A reader should therefore treat the comparison as holding the generator and the review set fixed, not as a strict single-variable manipulation. Retrieval differences would add noise rather than a systematic advantage to either arm, which makes a null easier to obtain, so this does not rescue the withdrawn positive result; it does mean the null is weaker evidence than a fully controlled design would give. The two arms average 105 and 97 words with distinct-1, the ratio of unique unigrams to total unigrams, at 0.736 and 0.760. They are therefore close on the surface features a rater could use to tell them apart, though the length difference is itself significant (paired Wilcoxon $p = 6 \times 10^{-8}$), which we return to when discussing what the lead author's preference tracks. Three raters scored all 200 blinded rows.

The IssueSpec does not measurably improve response quality. Pooled over the three raters the gain is +0.04 Likert points, 95% bootstrap CI $[-0.10, +0.18]$, paired Wilcoxon $p = 0.38$ on 100 pairs, Cliff's $\delta = 0.04$. Each review's two replies are shown in a randomised order, and we call the two display positions slots. The assignment came out 64/36 rather than balanced and the raters show a small preference for the first slot, so we also report a slot-stratified estimate, which moves the pooled figure to +0.03 and each rater's by at most 0.03. We use the unadjusted numbers throughout so that every figure in Table 14 is on one estimator, and release the stratified values with the data. Specificity behaves the same way, pooled -0.00 ($p = 0.87$), and the helpful rate is flat at 94% in both arms. On the evidence of this design we cannot claim that supplying the typed specification improves the response a user receives. We should be equally careful in the other direction. With 100 pairs and the observed spread, the smallest effect this design could detect at 80% power is about a fifth of a Likert point, so what we have ruled out is a large effect, not a small one. Detecting a gain the size of the one we measured would need on the order of 2,500 pairs. The honest statement is that the effect, if any, is small enough that this study cannot see it, not that it is zero.

The raters disagree with each other, and that is the interesting part. Table 14 breaks the result out. The lead author, who designed the system, prefers the IssueSpec arm decisively: $+0.56$ quality ($p = 2 \times 10^{-11}$, 59 wins, 5 losses, 36 ties) and $+0.39$ specificity ($p < 10^{-7}$). The two independent raters lean the other way, -0.29 and -0.15, neither significant at $\alpha = 0.05$ ($p = 0.06$ and 0.09). Inter-rater reliability is correspondingly low, quadratic-weighted κ of 0.03, 0.04 and 0.50, against 0.97 in the discarded template-based round. High agreement there was a symptom of the confound: when one arm is visibly broken, raters agree easily. Here they do not.

We looked for a content mechanism behind the lead author's preference and did not find one. Correlating each rater's per-review preference against the spec-derived features that the IssueSpec should introduce, named devices, reproduction steps, and component mentions, produces no correlation that survives inspection for any rater; the largest is $\rho = 0.19$ ($p = 0.06$) between the lead author's preference and reply length. So we can describe the disagreement but not explain it. Two readings remain open. The lead author may be perceiving triage value that a general reader has no reason to weigh, in which case the right evaluation asks developers rather than readers. Or the preference may be expectancy on the part of the person who built the system. This design cannot separate those, and we do not assert either.

Why we believe the raters this time, and what the check does and does not show. In the discarded round the two additional raters deviated from the lead author on only 16 and 19 of 400 rows, those deviation sets did not overlap on a single row, and every deviation was exactly one point. We initially read the zero overlap as the damning fact. It is not: with deviations that rare, the hypergeometric probability of zero overlap under perfect independence is 0.45, so zero overlap is the single most likely outcome and carries almost no information. The discriminating observation is the second one. On the quality column every deviation in both sheets had the same magnitude, one point, which is what a vector perturbed by a fixed step produces and not what human disagreement produces. We state the column because the specificity column in the same sheets does show magnitudes of one, two and three, so the signature is present in one column and not the other. In the reported round the raters differ from the lead author on 133 and 145 of 200 rows, the deviation sets overlap on 101 where chance predicts about 96, and the deviations take four distinct magnitudes from one to four points. We release the check with the data, and we state its weak prong plainly, because a check offered as a contribution should come with its own limits attached.

What survives. Less than we first thought. The claim that the typed IR carries downstream response quality does not survive, and neither does the retrieval finding we had treated as the surviving half: it was measured against the prompt baseline rather than a no-context arm, and against the no-context arm it is -0.07 at $p = 0.90$. Both sat in the discarded round. Stage 4 leaves us with a null and nothing else. Stage 3's contribution, producing tracker-ready structured records at corpus scale, stands on its own evidence in Section 5.1; it no longer has a demonstrated downstream payoff in response generation.

5.3. Aligning the IR-Aware Generator at Scale

Five policies were trained on distilGPT2 (82M parameters, 400 SFT samples, 296 KTO pairs, 100 DPO pairs): SFT-base, KTO [20], DPO [19], a constrained-proxy variant, and Lagrangian Constrained PPO [21, 22]. Head-to-head on a 100-review test set, per-policy BLEU-1 / ROUGE-L / BERTScore F1 are: SFT-base 0.090 / 0.097 / 0.802; KTO 0.068 / 0.074 / 0.792; DPO 0.084 / 0.088 / 0.800; **constrained-proxy 0.137 / 0.131 / 0.806**; Lagrangian PPO 0.087 / 0.091 / 0.798. The constrained proxy leads three of four metrics (BLEU-1, ROUGE-L, BERTScore F1; the rule-based safety-violation rate is uninformative here because compliance is satisfied at initialization), with +52% BLEU-1 over SFT-base. KTO underperforms here, consistent with its small-data assumption breaking on a 296-pair training set.

The dual-objective claim is not demonstrated, and we report that as the result. At distilGPT2 scale the compliance scorer is satisfied at initialization ($C = 0.94 \geq \tau$), so the Lagrange multiplier λ collapses to zero and the update reduces to single-objective optimization. Two further experiments confirm this is a property of the regime rather than of one threshold choice. A re-run that replaces the permissive scorer with the four operational compliance checks and tightens τ from 0.50 to 0.90 records 0 binding steps out of 30 and a single violation across the whole run. A sweep over $\tau \in \{0.0, 0.5, 0.7, 0.9, 1.0\}$ leaves the pass rate at 100% for four of the five policies at every threshold, with the first binding τ undefined for those four; only SFT-base ever dips, to 99% at $\tau \geq 0.9$. A model of this size cannot plausibly violate the rubric, so the constraint has no work to do and the CMDP machinery is untested under binding conditions. The +52% BLEU-1 figure for the constrained proxy is therefore a single-objective result, not evidence of a quality-versus-compliance trade-off, and we do not claim it as one.

The constraint binds once the policy can violate it. The obstacle is not parameter count, it is capability: distilGPT2 never produces the fluent developer-relations phrasing the rubric penalises, so it cannot violate and the constraint has nothing to hold back. We measured this directly. We first justified this by observing that 5.2% of our own Stage-4 generations trip the scorer. That figure is from the four withdrawn template-composed arms (21 violations in 400 replies). Scored the same way, the two model-generated arms we actually defend violate on 4 of 200 replies, 2.0%, and one of the four is an information-leak rather than an over-promise. The zero-shot Qwen2.5-1.5B probe gives 2 violations in 40 replies, 5.0%, whose 95% interval runs from roughly 1% to 17% and therefore cannot be distinguished from 2.0% either way. distilGPT2 gives roughly zero. The honest version of the argument is weaker than the one we first made: a 1.5B instruct model violates the rubric occasionally where distilGPT2 essentially never does, and that is enough for the constraint to bind, but we cannot claim it matches the rate of our own generations. We therefore re-ran the Lagrangian loop with Qwen2.5-1.5B-Instruct as the policy, LoRA adapters over the attention projections, the frozen base as the KL reference, and a zero-tolerance threshold $\tau = 1.0$; the scorers, the dual-ascent rule, and the batch size are unchanged from the distilGPT2 run. The behaviour inverts. Over 90 steps the constraint binds on 13 of them (14.4%), and λ leaves zero, rises monotonically to 0.65, and never collapses back. A 30-step run under the same setup reaches $\lambda = 0.125$, so the multiplier scales with the horizon rather than saturating. Split by thirds, the trajectory shows the dual ascent responding to violations as they arrive: 1 binding step and $\lambda = 0.05$ in the first thirty, 7 and $\lambda = 0.40$ in the second, 5 and $\lambda = 0.65$ in the third. The qualitative claim that the CMDP degenerates to single-objective optimization is therefore a property of the proof-of-concept policy, not of the formulation.

What is and is not shown. The two series move in opposite directions across the run, quality from 0.565 to 0.590 and compliance from 0.997 to 0.989, while λ climbs. We are careful not to call this a trade-off. The step-level correlation between quality and compliance is $\rho = -0.12$ ($p = 0.25$), so what we have is two separate trends over a single run and no measured coupling between them, which is why the abstract reports the trade-off as undemonstrated. What we cannot yet show is the other half of the story, the multiplier growing large enough to pull the policy back into compliance. At a 5% base violation rate and a batch of four, roughly one violating sample arrives every seven steps, and 90 steps of LoRA updates at this learning rate

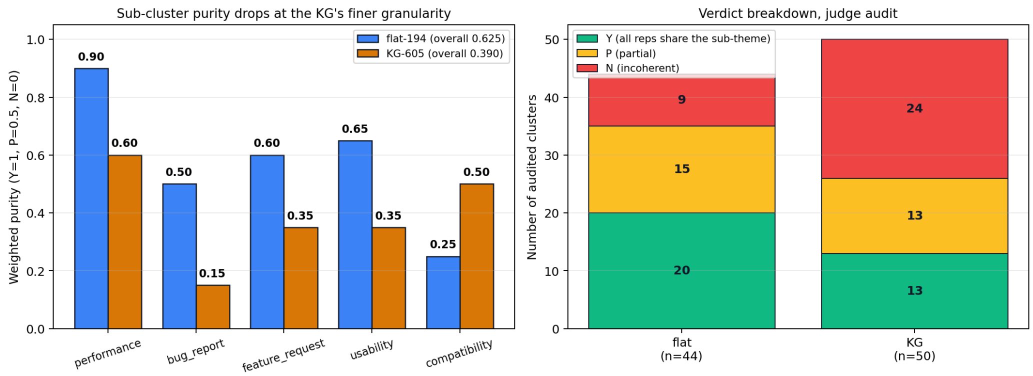

Figure 3: Extrinsic cluster purity for the flat baseline and the KG hierarchy, from the saved judge audit. *Left:* weighted purity per issue type, flat-194 against KG-605. The KG loses on four of five types, which is the finer-granularity cost we report rather than a result in its favour. *Right:* the underlying Y/P/N verdict counts behind the two overall numbers, 0.625 for flat and 0.390 for the KG. This figure is the evidence for the A2 ablation and for the claim in Section 6.1 that the KG's contribution is prioritization, not cluster quality.

do not accumulate enough constraint pressure to reverse the drift. So the correct statement is narrower than a confirmed RQ3 and wider than the earlier null: the dual objective engages, the multiplier responds monotonically to violations, and the tension between the objectives is measurable, while closing the loop back to compliance needs a longer horizon or a higher base violation rate than a 1.5B instruct model supplies.

What this layer contributes is the CMDP formulation instantiated over four domain-specific dev-rel constraints, an end-to-end trainer that runs, the threshold sweep that establishes the non-binding regime, the capability probe that explains it, the binding-constraint re-run, and the released testbed.

5.4. Cluster Quality and Purity

Stage 2 was redesigned from a flat UMAP+HDBSCAN baseline (194 clusters, mean size 471.0) to the three-layer KG-hierarchical design (605 sub-clusters, mean size 15.9), both over the 8,404-review sample the knowledge graph is built on. We compute two complementary validity axes. *Intrinsic geometry*, Davies-Bouldin [58], Calinski-Harabasz [59], silhouette [60], follows standard IR/clustering practice [61, 62, 63]. *Extrinsic cluster purity* [64, 61] uses a Y/P/N weighted-purity rubric, $\text{purity}_w = (|Y| + 0.5|P|)/(|Y| + |P| + |N|)$, with Y = all 5 share theme, P = 3–4 of 5 share, N = fewer than 3 share, following inter-rater conventions compatible with Landis-Koch [51].

We reported that KG-hierarchical achieves 5.4× lower Davies-Bouldin and 1.9× higher Calinski-Harabasz than the flat baseline. We withdraw those two figures. The script that produced the flat baseline's intrinsic metrics draws its cluster labels as a random subsample of 10,000 reviews spanning all 194 clusters, but collects the review texts to embed by walking the clusters in order, and the first flat cluster alone contains 10,507 reviews. The embeddings therefore come from a single cluster while the labels span 194, so the flat column's DB of 12.15 and CH of 0.98 measure random labelling rather than the flat partition. The KG column was computed on a correctly paired set, so the comparison was a valid measurement against a broken one. Table 15 keeps the KG column and marks the flat intrinsic column as withdrawn. Repairing the alignment does not make the two columns comparable in any case: the flat side embeds 10,000 full reviews while the KG side embeds three representative reviews per cluster, so the two are computed over different kinds of object. That asymmetry is a second reason not to reinstate the comparison, and the count-matched A1b ablation, which embeds representatives on both sides, is the only intrinsic comparison we report. The count-controlled A1b ablation below is unaffected, uses a separate correctly paired script, and is what our Stage-2 conclusion now rests on. Extrinsic purity is also unaffected: the KG's finer clusters score 0.39 against the flat baseline's 0.625 under the same Qwen judge. By Landis-Koch [51] convention adapted to weighted purity, the flat baseline's 0.66 lead-author purity sits in the substantial-agreement band (≥ 0.61) and KG-605's 0.39 in the fair band (≥ 0.21), reflecting the granularity-vs-coherence trade-off the A1b ablation makes explicit.

Count-controlled diff from original. A count-controlled ablation (A1b) disambiguates two evaluation axes that prior cluster-quality work tends to conflate: intrinsic geometry and practitioner-side prioritization (centrality-weighted coverage). On 1,815 representative reviews (3 per sub-cluster across 605 sub-clusters), an agglomerative clustering forced to 605 clusters reaches DB = 1.12, CH = 4.21, silhouette = +0.148, against KG-605's 2.24 / 1.85 / −0.234. This is now the only intrinsic-geometry comparison we make, since the flat-194 figures are withdrawn above: on geometry, count-matched agglomerative clustering beats the KG on all three metrics.

Where the KG actually wins. On the prioritization axis the story reverses. Of the 10,534 aspect nodes in the KG, the top five by PageRank are touched by 55% of reviews and the top ten by 66%. We give the deduplicated figures; counting a review once per aspect it mentions inflates these to 74% and 96%, and that inflated pair should not be used. A developer triaging in PageRank order completes three-quarters of the complaint volume after reading five aspect buckets, whereas random aspect ordering would require thousands of buckets to reach the same coverage, a roughly $10^3\times$ concentration ratio. The KG additionally enables per-aspect drill-down, distinguishing `battery`→Samsung-drain from `battery`→fast-charge.

5.5. Ablations

An ablation removes one component and re-measures, so that a component's contribution is demonstrated rather than assumed. Eight are listed below, counting A1b, which a reviewer asked for. Seven were run and one, A7, is deferred for a reason given at the end. Each one below states what it removes, what the removal is supposed to test, what happened, and what we conclude, because three of the seven came back against the design and a summary line would hide that. Table 16 collects the verdicts.

A1, remove the knowledge graph and feed flat clusters into Stage 3. This asks whether Stage 3 reads the graph or merely the group. Stage 3 quality does not change, because the translator consumes a cluster centroid and its representative reviews, not the graph edges. The KG is therefore not a Stage 3 dependency, which narrows where its value could lie.

A1b, compare the KG against flat clustering matched to the same number of clusters. This is the reviewer-prompted version of A1 and it exists because of a confound. The KG produces 605 sub-clusters and the flat baseline produces 194, and intrinsic metrics move with cluster count on their own, so any comparison between the two conflates structure with granularity. Re-running agglomerative clustering forced to exactly 605 clusters removes the confound. The flat version then wins on all three metrics, Davies-Bouldin 1.12 against 2.24 (lower is better), Calinski-Harabasz 4.21 against 1.85 (higher is better), and silhouette $+0.148$ against -0.234 (higher is better, and a negative value means the average point sits closer to a neighbouring cluster than to its own). The honest conclusion is that the KG does not produce geometrically better clusters. What it produces is an ordering, through PageRank centrality over the review-aspect graph, and prioritization is the only cluster-layer claim we make.

A2, remove hierarchical sub-clustering. Weighted Y/P/N purity, judged on five representative reviews per cluster, is 0.625 for the flat 194 and 0.390 for the KG's 605. Finer granularity costs coherence here rather than buying it. Taken with A1b this is the second result against the cluster layer, and we report both.

A3, remove taxonomy grounding. The LLM writes a free-form issue description instead of filling a routed template. Strict template-fill falls from 0.96 to 0.69, and the per-type structural columns fall to exactly zero, because an unrouted generator never emits a reproduction-step list or an As-a/I-want/So-that triple at all when nothing asks it to. This is the ablation that isolates the paper's central mechanism. The gain is not the language model, it is the routing.

A4, remove the HITL-2 spec review. Benchmark scores do not move, and the reason is procedural rather than substantive. HITL-2 is a production-time gate; the 100 specs used in the evaluation were scored before any human correction, so the headline numbers already describe pre-HITL output. The ablation therefore says nothing about whether HITL-2 helps in deployment, and we do not claim that it does.

A5, remove retrieval and keep the IssueSpec. BLEU-1 moves by -0.006, ROUGE-L by -0.008, and BERTScore F1 by -0.004, all under one percent absolute. Three caveats belong with this result. The first is that A5 was generated by the template composer that Section 5.2 withdraws, not by the model-generated pipeline, so it measures retrieval's effect on a generator we no longer report. First, A5 was scored on automatic metrics only, while A6 was scored by humans, so the pair is not measured on one instrument and the contrast between them is directional rather than a like-for-like comparison. A human-rated no-retrieval condition is the missing cell. Second, we withdrew the companion claim that retrieval without the IR scores below a no-context baseline. That comparison used the prompt baseline, which is not a no-context condition, and against the arm that is, the difference is -0.07 at $p = 0.90$. So A5's reading is simply that retrieval makes no measurable difference here, with no accompanying story about it doing harm.

A6, remove the IssueSpec and keep retrieval. This ablation is superseded. Under the confounded design it showed quality falling from 4.59 to 2.24; under the corrected same-model design (Section 5.2) removing the IssueSpec changes quality by -0.04, which is not distinguishable from zero. A5 and A6 together now say something different from what we first concluded: neither retrieval nor the typed intermediate produces a measurable change in the reply a user receives, and the Stage 3 to Stage 4 coupling we claimed is not supported.

A7, single-stream RLHF feedback. Not run. It was designed to test whether quality and compliance feedback can share one channel, which only becomes a meaningful question once the constraint binds and the two signals actually compete. Section 5.3 reaches the first half of that condition and not the second, so A7 stays deferred rather than reported.

6. Discussion

6.1. What Carries the Result, and What Does Not

A five-stage pipeline invites the question of whether every stage earns its place. Our own ablations say it does not, and separating the two groups is more useful to a reader than defending the whole.

Load-bearing. Two components carry the measured gains. Type-routed templating is the principal driver of Stage 3 quality: removing taxonomy grounding (A3) drops strict template-fill from 0.96 to 0.69, and the structural per-field metrics collapse to zero because an unrouted generator does not emit the reproduction-step list or the As-a/I-want/So-that triple at all. By contrast the typed IR is *not* a demonstrated driver of Stage 4 quality: under a same-model comparison removing it moves quality by 0.04 points ($p = 0.38$, Section 5.2), and removing retrieval instead (A5) moves every automatic metric by less than one point absolute. Our earlier reading of the retrieval arm does not survive either, and the error was ours to make twice. We first reported that retrieval alone was indistinguishable from a no-context baseline, then corrected that to significantly worse. Both statements compared the retrieval arm against the *prompt* baseline, which is not a no-context condition; it carries a developer-relations instruction. Against the actual no-context arm, which sees the review text only, retrieval scores 2.24 against 2.31, a difference of -0.07 at $p = 0.90$. And all three arms come from the discarded template-composer round, so none of them supports a claim in any direction. Nothing about retrieval survives from Stage 4 beyond the null.

Not load-bearing, and reported as such. The knowledge graph does not win on intrinsic cluster geometry. Under count matching (A1b), flat agglomerative clustering beats the KG on all three intrinsic metrics, and sub-cluster purity is lower at the KG's finer granularity (A2). The KG's defensible contribution is centrality-based prioritization, that is, which issues a developer should see first, not tighter clusters; we make no cluster-quality claim for it. We should be straight about how far even that goes. PageRank runs on the bipartite review-aspect graph, and nothing in our results shows it needs the hierarchical sub-clustering layer to do so. A flat clustering with the same aspect graph would plausibly support the same ranking, and we did not test that configuration. Stage 2 as built is therefore justified in its prioritization role and unjustified in its sub-clustering role, and a reader designing a lighter system should read it that way. Stage 5 sits in between. As Section 5.3 reports, the compliance constraint does bind once the policy is capable of violating it, and the dual update engages rather than collapsing, but no quality-versus-compliance trade-off is demonstrated at the scale we can train at. It is released as a testbed with a partial result. SpecCov falls in the same group and for a sharper reason: tested against human faithfulness judgements it shows no correlation at all (Section 5.1), so we withdraw it as a measure rather than defend it.

What this means for the framework claim. The contribution that survives ablation is the typed IR and the type-routed templating that produces it, together with the verified-anchor procedure that makes the IR producible from noisy labels at corpus scale. Stages 2 and 5 are supporting infrastructure with honestly mixed evidence. A reader who wants the smallest system that reproduces the headline result needs Stage 1, Stage 3, and a generator conditioned on the resulting specification. That is less than Stage 4 as we built it: A5 shows the retrieval layer inside Stage 4 contributes under one percent on automatic metrics, so the part of Stage 4 that earns its place is the conditioning, not the retrieval.

6.2. Implications

Three claims travel beyond app reviews. *First*, LLM annotation noise on software-engineering text is not uniform, but the shape is not the one we first reported. We had described the 25% mislabelling rate as concentrated on minority classes. Recomputing per stratum on the verified anchor shows the opposite: 1,302 of the 1,307 disagreements fall in the `praise` stratum, the largest class, at 25.8%, against 0.5% on `performance`. What is true, and narrower, is that the labeller never emits two of its seven classes at all across the corpus, 8 `compatibility` and 184 `performance` labels in 215,583 reviews. Systematic absence of a class, rather than elevated error on rare classes, is what a small verified anchor is able to repair. *Second*, a caution rather than a claim. We expected a typed structural intermediate to be what does the load-bearing work in spec-aware response generation, and reported it as such. Section 5.2 withdraws that. The only downstream movement we can still point to is +0.12 on a rule-based scorer from agentic critique-revise iteration, on ten reviews, which is too small a sample to build on. What generalises is the negative lesson: a structured intermediate can be demonstrably better as an artifact and still leave the generated text indistinguishable, and pipelines of this shape should measure the two separately. *Third*, the count-controlled cluster-validation ablation (A1b) delivers a self-contained methodological contribution: whenever a new clustering design changes cluster count and structure simultaneously, intrinsic geometry metrics (DB, CH, silhouette) conflate the two effects. The minimum reporting standard our analysis suggests is to (a) compare against a count-matched baseline on the same intrinsic metrics, and (b) pair that with a domain-utility metric (PageRank coverage in our case). This pattern applies directly to clustering-design comparisons in IR, software engineering, and NLP.

6.3. Threats to Validity

Construct. Template-fill measures coverage rather than content correctness, and Section 5.1 shows that SpecCov does not fill that gap: it is a lexical-grounding proxy that does not track human faithfulness judgement [47, 48, 49, 50]. The paper therefore has no validated content-correctness measure for Stage 3, and the five-dimension rubric carries that burden alone. NLI- and QA-based faithfulness measurement [55, 56, 57] is the clear next step. *Internal.* The count-versus-structure confound is exposed and resolved by the A1b ablation, which reframes the knowledge graph's contribution as centrality-based prioritization rather than intrinsic geometry. A second internal threat we did not remove is that the 100 clusters carrying the Stage 3 and Stage 4 results were produced by the KG pipeline, the same layer A1b and A2 find wanting. Since flat clustering scores higher on purity (Figure 3), the headline gains may be measured on a slightly worse partition than a simpler system would produce, which would understate rather than inflate them. Re-running Stages 3 and 4 over flat clusters would settle it and is the cheapest open experiment in the paper. *External.* RRGen contains 58 anonymized Android apps; V5 transfers zero-shot to the Maalej label space. Stage 2–4 cross-corpus transfer remains to be validated. The headline LLM is Claude Opus, with Llama-3.3-70B and Qwen2.5-{3B, 1.5B} as cross-family bias controls; GPT-4o and Gemini-1.5-Pro replication is queued (Section 6.5). *Conclusion.* The three-rater agreement on the classification gold ($\kappa = 0.968$ on the Y/N verdict) sits well above what free-labeling studies of app reviews report, which invites the question of whether the raters were genuinely independent. Three properties of the design account for the gap. The raters verify a supplied label rather than choose one from seven, which collapses most of the decision space; a 20-review calibration round with adjudicated disagreements precedes the main task; and the written protocol fixes eight boundary rules that carry the cases annotators most often split on. Residual disagreement is rater-specific rather than random, one rater accounts for most of the `usability` reassignments, which is the pattern independent annotation produces. Stage 4 sits differently. All three raters scored the complete 200-row re-run, and the gain does *not* reproduce per rater: the lead author measures +0.56 and the two independent raters −0.29 and −0.15. That disagreement, not the pooled null, is the honest summary of what we know about Stage 4. *Reliability, and which α belongs to what.* Three agreement statistics appear in this paper and they measure different things, so a single "moderate α" reading would be wrong. The classification gold is annotated by three humans and reaches Fleiss $\kappa = 0.968$ on the verdict and 0.979 on the seven-class label. The Stage-4 response set that we report is rated by the same three humans on all 200 rows, with quality weighted κ of 0.03, 0.04 and 0.50. That is low, and it is the number to quote: the 0.970 to 0.986 range belongs to the discarded round, where one arm was visibly broken and agreement was easy. The one moderate figure, $\alpha = 0.451$, comes from a different and deliberately weaker panel: a cluster-purity check whose three raters are the lead author plus two Qwen2.5-3B prompts, following the LLM-as-rater convention of Gilardi et al. [44]. It bounds how far an LLM rater tracks a human on a subjective sub-theme judgement; it is not the reliability of any headline result, and no claim in this paper rests on it alone. *Statistical.* Experiment 2 now reports a null, so the relevant question is power rather than correction for multiplicity. With 100 paired reviews and the observed per-review spread, the 95% bootstrap interval $[-0.10, +0.18]$ excludes effects larger than about a fifth of a Likert point in either direction. We can therefore rule out a large effect, not a small one; an effect of a tenth of a point would need several times this sample.

6.4. Reproducibility and Extensibility

Every numerical claim in Section 5 is backed by a saved data file and runs end-to-end from raw RRGen to Stage 4 output in approximately six hours on a single GPU-equipped workstation. The release bundles:

- V1–V5 classifier checkpoints + cleanlab correction procedure;

- the 5,230-record verified anchor, which covers 5,229 distinct corpus rows and 490-review classification gold;
- both Stage-4 rating rounds, the discarded 400-row one and the reported 200-row re-run, together with the same-model generated replies and the rating-provenance check that distinguishes them;
- the count-controlled A1b output;
- the SpecCov scorer plus the 60-spec, three-rater human faithfulness ratings and the validation analysis;
- the CMDP-RLHF testbed (KTO, DPO, constrained-proxy, and Lagrangian Constrained PPO trainers);
- the 100-cluster benchmark, with the 99-review LLM-assisted purity subsample ($\alpha = 0.451$) kept separate from the all-human agreement sets;
- a pinned-environment Docker image (single-command build on any CUDA 12.x host);
- a nineteen-segment verifier that recomputes the numerical claims in Section 5 from the saved data files, including the corrected Stage-4 re-run, the three-rater agreement statistics, the cross-family rubric, the SpecCov validation, the binding-constraint run, and the generation-variance sweep. Two tables are not covered by it, the per-class F1 breakdown and the multi-model template-fill row, and we say so rather than claim complete coverage. The discarded Stage-4 round retains its own segment, labelled as withdrawn.

The pipeline is composable by construction: because each stage emits a typed intermediate, components (classifier, clusterer, taxonomy template, alignment trainer) can be substituted independently without breaking downstream contracts. Future researchers can adopt or replace any single component while keeping the rest of the IR-centered evaluation harness intact. Deterministic components (clustering, classifier training) use fixed seeds; the LLM-dependent stages (annotation, IR generation, and response generation) reproduce statistically rather than bit-identically. We measured how large that sampling variance is. Regenerating Stage 3 specs three times over the same 15 stratified clusters with the local Qwen2.5-3B generator at temperature 0.7, changing only the seed, gives strict template-fill 0.685, 0.700, and 0.634 (mean 0.673, sd 0.029, range 0.067) and SpecCov 2.73, 2.67, and 3.00 (mean 2.80, sd 0.144, range 0.333), with no JSON parse failures in any run. Run-to-run noise on template-fill is therefore around three points on a 0–1 scale for this generator and prompt. That is the yardstick a reader should hold up against the differences we report. The gaps the paper rests on clear it comfortably: 0.96 versus 0.69 for taxonomy grounding, and 0.96 versus 0.48 across the model families. The withdrawn Stage-4 effect illustrates the opposite case, and is worth stating in these terms: at +0.04 it sits an order of magnitude below this noise floor, which is another way of saying there is nothing there to measure. Differences of a few hundredths between adjacent conditions, such as the 4.09 versus 3.96 at the top of the cross-family rubric, do not clear it and should not be read as a stable ordering on their own. Two limits remain: the headline Claude generations were not repeated, because that needs API budget rather than new code, and Stage 4 response generation was not repeated either. The released script runs both sweeps unchanged.

6.5. Future Work

Five extensions map directly to the limitations above; each starts from a resource released with this paper.

(0) Replace SpecCov and widen the cross-family comparison. The SpecCov validation in Section 5.1 came back negative, so Stage 3 needs a faithfulness measure that actually works. The natural replacement is entailment-based: score each spec field against the source reviews with an NLI model [55, 56] or a question-generation-and-answering pipeline [57], then validate that measure against the same 60-spec human ratings we release here, which now serve as a ready-made benchmark. Second, the cross-family rubric comparison (Table 8) rests on 14 matched bug-report clusters for the four-way panel, because that is the extent of the overlap between the four generators' spec sets, and it is scored by a judge from one of the four families. Generating the missing spec types for the two Qwen models and re-scoring with a judge outside all four families would remove both limits. Both are runs rather than redesigns.

(1) Ask the right raters, then scale. The Stage 4 re-run carries three annotators on all 200 rows and returns a null with the raters disagreeing in direction. Before growing the sample, the design should change: the reply from a triage pipeline is consumed by a developer deciding what to do, not by a general reader, so the next round should recruit raters with software-engineering experience and no stake in the system. Scaling from 100 clusters to 200–300, with three annotators per IssueSpec and Krippendorff's α at the IssueSpec level, follows that rather than precedes it.

(2) CMDP RLHF at a scale where the trade-off shows. The binding-constraint re-run in Section 5.3 establishes that the dual objective engages; what it does not show is a trade-off, because a 5% base violation rate over 30 steps supplies too little signal. The next run wants a longer horizon and a policy with a higher base violation rate, on an 8B-class model. *Original plan.* Where the compliance constraint is expected to bind, activating the Lagrange multiplier and making the Pareto-dominance comparison meaningful.

(3) Frontier-family replication. GPT-4o and Gemini-1.5-Pro on Stages 3–4. The current paper closes the Meta-family slice with Llama-3.3-70B and the Alibaba slice with Qwen2.5-{3B, 1.5B}.

(4) Cross-corpus transfer. Stages 2–4 on additional review corpora; V5 already transfers zero-shot to Maalej [1].

(5) Agentic Planner extension to Stage 4. The Planner dispatches a different sub-agent profile per issue type: defect-repair for `bug_report` via SWE-Agent [11] or RepairAgent [12], scaffolding for `feature_request`, profiler-driven optimization for `performance`, a Nielsen-heuristic checker for `usability`, and a device-OS test-matrix executor for `compatibility`. A spec-level stub already runs the dispatch across all five issue types; end-to-end patch evaluation on real repositories through HyperAgent [30], measured by patch acceptance and test-pass rate, is the next step.

7. Conclusion

We proposed **IssueSpec**, a knowledge-graph-grounded typed intermediate representation that closes the five existing research gaps identified in Section 1: classification stops at the label, response generation skips the structured intermediate, agentic software engineering assumes the spec exists, knowledge graphs emit no typed schema, and RLHF has not addressed dev-rel compliance. We set out to show that the typed IR is the load-bearing component of spec-aware response generation, and we cannot. An isolation experiment that holds the generator fixed moves human-rated quality by +0.04 Likert points ($p = 0.38$, three raters, 100 paired reviews), and the raters disagree in direction. The +2.35 we previously reported came from a comparison whose two arms were written by different template composers. Nor does the negative half of our original hypothesis survive. We had reported that retrieval alone falls below a no-context baseline; against the arm that actually has no context it scores 2.24 against 2.31 at $p = 0.90$, and that comparison sits inside the discarded round in any case. Stage 4 yields one usable result, which is the null. A verified-anchor procedure makes the IR producible at corpus scale from noisy LLM labels, lifting Cohen's κ from 0.163 to 0.592, and to 0.616 on the strictly held-out gold, with 88.66% classifier endorsement of corrections. Cross-family replication on four LLMs (Claude, Llama, Qwen-3B, Qwen-1.5B) preserves the rank order, ruling out single-model bias. Two findings travel beyond app reviews: an LLM labeller on software-engineering text can omit whole classes rather than merely err on them, and that a structured intermediate can be demonstrably better as an artifact while leaving the text generated from it indistinguishable, a distinction that pipeline evaluations of this shape should measure separately. We release the IssueSpec schema, the verified anchor, the SpecCov scorer together with the human ratings that show it does not measure faithfulness, the count-controlled cluster-validation ablation, the structured-RAG architecture, the CMDP-RLHF trainers, and the benchmark dataset to support follow-on research.

GenAI Usage Disclosure

We disclose all uses of generative AI in this work in line with Elsevier policy on the use of generative AI in scientific writing. *Writing and code assistance.* An earlier version of this disclosure said that Claude assisted only with grammatical refinement, caption polishing, and LaTeX formatting, and that all pipeline code, experimental scripts, and analysis were written by the human authors. That understated the use, and the repository we release contradicts it: 38 of the 47 commits carry a `Co-Authored-By: Claude` trailer, including commits that add pipeline stages, the constrained-PPO trainer, the analysis scripts, and the correction passes described in Section 5.2. The accurate statement is that Anthropic's Claude was used throughout implementation and analysis as a coding assistant, under author direction and review, and also for manuscript editing. Every design decision, every experimental choice, and every claim in this paper is the authors' own, and the numbers were produced by running the released code on the released data. We prefer to correct this here rather than leave a disclosure that the artifact refutes. Every structural, framing, experimental-design, and claim decision was made by the authors, and every reported number was produced by author-written code run under human supervision and verified against saved data files. *Models inside the system.* Several generative models are themselves pipeline components and are reported as such in the body: Claude (Opus 4.7) is the headline Stage 3 generator (Table 11); Llama-3.3-70B-Instruct (via Groq) and Qwen2.5-{3B, 1.5B}-Instruct are cross-family Stage 3 comparators (Table 9); Qwen2.5-3B-Instruct is the LLM-as-judge for the five-dimension rubric and the Y/P/N cluster-purity audit, both calibrated against the lead author (Tables 10, 15). The V2 corpus labels in Section 5.1 are LLM-generated and explicitly treated as noisy, which is precisely what motivates the verified-anchor procedure of Stage 1. *Human verification.* No reported numerical result, expert annotation, or ground-truth label was automatically generated by AI without human verification. The 5,230-review verified anchor, the 490-review classification gold, the 99-review three-rater purity ratings, and the 20 lead-author IssueSpec references were all produced and reviewed by human annotators (Section 4).

CRediT authorship contribution statement

Fabiha Jalal: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review and editing, Visualization. **Sadia Tasnim Dhruba:** Investigation, Data curation, Validation, Writing – review and editing. **Tazrian Zaman Tushti:** Investigation, Data curation, Validation, Writing – review and editing. **Ajwad Abrar:** Methodology, Supervision, Writing – review and editing. **Md Kamrul Hasan:** Supervision, Resources, Writing – review and editing. **Hasan Mahmud:** Supervision, Resources, Project administration, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All code, configuration files, and analysis scripts are available in the project repository. A verification bundle that reproduces every numerical claim in this paper is archived on Zenodo (DOI 10.5281/zenodo.21982774, v2.0), and the Stage-1 V5 classifier is released on Hugging Face. That bundle supersedes an earlier release (DOI 10.5281/zenodo.20320410) which predates the Stage-4 correction described in Section 5.2; both versions remain resolvable, and the earlier one should not be used. The underlying review corpus derives from the publicly released RRGen dataset of Gao et al.; derived artifacts are redistributed subject to the terms of that original release.

Acknowledgements

We thank the human annotators who produced the verified anchor and the lead-author reference set. The authors verified all technical content, experimental results, and claims in this paper.

References

[1] W. Maalej, H. Nabil, Bug report, feature request, or simply praise? on automatically classifying app reviews, in: 2015 IEEE 23rd international requirements engineering conference (RE), IEEE, 2015, pp. 116–125.

[2] C. Gao, J. Zeng, X. Xia, D. Lo, M. R. Lyu, I. King, Automating app review response generation, in: 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2019, pp. 163–175.

[3] J. Dąbrowski, E. Letier, A. Perini, A. Susi, Analysing app reviews for software engineering: a systematic literature review, Empirical Software Engineering 27 (2) (2022) 43.

[4] O. Chaparro, J. Lu, F. Zampetti, L. Moreno, M. Di Penta, A. Marcus, G. Bavota, V. Ng, Detecting missing information in bug descriptions, in: Proceedings of the 2017 11th joint meeting on foundations of software engineering, 2017, pp. 396–407.

[5] T. Zimmermann, R. Premraj, N. Bettenburg, S. Just, A. Schroter, C. Weiss, What makes a good bug report?, IEEE Transactions on Software Engineering 36 (5) (2010) 618–643.

[6] N. Chen, J. Lin, S. C. Hoi, X. Xiao, B. Zhang, Ar-miner: mining informative reviews for developers from mobile app marketplace, in: Proceedings of the 36th international conference on software engineering, 2014, pp. 767–778.

[7] L. Villarroel, G. Bavota, B. Russo, R. Oliveto, M. Di Penta, Release planning of mobile apps based on user reviews, in: proceedings of the 38th International Conference on Software Engineering, 2016, pp. 14–24.

[8] A. Di Sorbo, S. Panichella, C. V. Alexandru, C. A. Visaggio, G. Canfora, Surf: Summarizer of user reviews feedback, in: 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C), IEEE, 2017, pp. 55–58.

[9] C. Gao, W. Zhou, X. Xia, D. Lo, Q. Xie, M. R. Lyu, Automating app review response generation based on contextual knowledge, ACM Transactions on Software Engineering and Methodology (TOSEM) 31 (1) (2021) 1–36.

[10] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al., Retrieval-augmented generation for knowledge-intensive nlp tasks, Advances in neural information processing systems 33 (2020) 9459–9474.

[11] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, O. Press, Swe-agent: Agent-computer interfaces enable automated software engineering, Advances in Neural Information Processing Systems 37 (2024) 50528–50652.

[12] I. Bouzenia, P. Devanbu, M. Pradel, Repairagent: An autonomous, llm-based agent for program repair, in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), IEEE, 2025, pp. 2188–2200.

[13] A. De Vink, N. Amat-Lefort, L. Han, Reviewgraph: A knowledge graph embedding based framework for review rating prediction with sentiment features, in: 2025 IEEE International Conference on Knowledge Graph (ICKG), IEEE, 2025, pp. 43–50.

[14] Z. Yu, S. Wu, J. Jiang, D. Liu, A knowledge-graph based text summarization scheme for mobile edge computing, Journal of Cloud Computing 13 (1) (2024) 9.

[15] S. Keertipati, B. T. R. Savarimuthu, S. A. Licorish, Approaches for prioritizing feature improvements extracted from app reviews, in: Proceedings of the 20th international conference on evaluation and assessment in software engineering, 2016, pp. 1–6.

[16] K. Kaur, P. Kaur, BERT-RCNN: An automatic classification of app reviews using transfer learning based RCNN deep model, Research Square preprint (2023).

[17] F. A. Shah, K. Sirts, D. Pfahl, Mining user opinions to support requirement engineering: An empirical study, in: Advanced Information Systems Engineering (CAiSE 2020), Vol. 12127 of Lecture Notes in Computer Science, Springer, 2020, pp. 401–416.

[18] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan, Swe-bench: Can language models resolve real-world github issues?, arXiv preprint arXiv:2310.06770 (2023).

[19] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, C. Finn, Direct preference optimization: Your language model is secretly a reward model, Advances in neural information processing systems 36 (2023) 53728–53741.

[20] K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, D. Kiela, Kto: Model alignment as prospect theoretic optimization, arXiv preprint arXiv:2402.01306 (2024).

[21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv preprint arXiv:1707.06347 (2017).

[22] J. Dai, X. Pan, R. Sun, J. Ji, X. Xu, M. Liu, Y. Wang, Y. Yang, Safe rlhf: Safe reinforcement learning from human feedback, in: International Conference on Learning Representations, Vol. 2024, 2024, pp. 50750–50777.

[23] C. Northcutt, L. Jiang, I. Chuang, Confident learning: Estimating uncertainty in dataset labels, Journal of Artificial Intelligence Research 70 (2021) 1373–1411.

[24] E. Altman, Constrained Markov decision processes, Routledge, 2021.

[25] Y. Geifman, R. El-Yaniv, Selective classification for deep neural networks, Advances in neural information processing systems 30 (2017).

[26] G. Bansal, B. Nushi, E. Kamar, W. S. Lasecki, D. S. Weld, E. Horvitz, Beyond accuracy: The role of mental models in human-ai team performance, in: Proceedings of the AAAI conference on human computation and crowdsourcing, Vol. 7, 2019, pp. 2–11.

[27] E. Guzman, W. Maalej, How do users like this feature? a fine grained sentiment analysis of app reviews, in: 2014 IEEE 22nd international requirements engineering conference (RE), Ieee, 2014, pp. 153–162.

[28] A. Asai, Z. Wu, Y. Wang, A. Sil, H. Hajishirzi, Self-rag: Learning to retrieve, generate, and critique through self-reflection, in: International conference on learning representations, Vol. 2024, 2024, pp. 9112–9141.

[29] O. Khattab, K. Santhanam, X. L. Li, D. Hall, P. Liang, C. Potts, M. Zaharia, Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive nlp, arXiv preprint arXiv:2212.14024 (2022).

[30] H. N. Phan, T. N. Nguyen, P. X. Nguyen, N. D. Bui, Hyperagent: Generalist software engineering agents to solve coding tasks at scale, arXiv preprint arXiv:2409.16299 (2024).

[31] M. A. Hearst, Untangling text data mining, in: Proceedings of the 37th Annual meeting of the Association for Computational Linguistics, 1999, pp. 3–10.

[32] S. Sarawagi, Information extraction, Foundations and trends in databases 1 (3) (2008) 261–377.

[33] M. E. Joorabchi, A. Mesbah, P. Kruchten, Real challenges in mobile app development, in: 2013 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, IEEE, 2013, pp. 15–24.

[34] M. Hu, B. Liu, Mining and summarizing customer reviews, in: Proceedings of the tenth ACM SIGKDD international conference on Knowledge discovery and data mining, 2004, pp. 168–177.

[35] N. Reimers, I. Gurevych, Sentence-bert: Sentence embeddings using siamese bert-networks, in: Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international

joint conference on natural language processing (EMNLP-IJCNLP), 2019, pp. 3982–3992.

[36] L. McInnes, J. Healy, J. Melville, Umap: Uniform manifold approximation and projection for dimension reduction, arXiv preprint arXiv:1802.03426 (2018).

[37] L. McInnes, J. Healy, S. Astels, et al., hdbscan: Hierarchical density based clustering., J. Open Source Softw. 2 (11) (2017) 205.

[38] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, Computer networks and ISDN systems 30 (1-7) (1998) 107–117.

[39] L. Page, S. Brin, R. Motwani, T. Winograd, The PageRank citation ranking: Bringing order to the web, Tech. Rep. 1999-66, Stanford InfoLab (1999).

[40] M. Cohn, User stories applied: For agile software development, Addison-Wesley Professional, 2004.

[41] M. Wynne, A. Hellesoy, S. Tooke, The Cucumber Book: Behaviour-Driven Development for Testers and Developers, 2nd Edition, Pragmatic Bookshelf, 2017.

[42] I. O. for Standardization, S. Technical Committee ISO/IEC JTC 1, Information technology. Subcommittee SC 7, systems engineering, Systems and Software Engineering: Systems and Software Quality Requirements and Evaluation (SQuaRE): System and Software Quality Models, ISO, 2011.

[43] J. Nielsen, Usability engineering, Morgan Kaufmann, 1994.

[44] F. Gilardi, M. Alizadeh, M. Kubli, Chatgpt outperforms crowd workers for text-annotation tasks, Proceedings of the National Academy of Sciences 120 (30) (2023) e2305016120.

[45] N. Pangakis, S. Wolken, N. Fasching, Automated annotation with generative ai requires validation, arXiv preprint arXiv:2306.00176 (2023).

[46] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, et al., Judging llm-as-a-judge with mt-bench and

chatbot arena, Advances in neural information processing systems 36 (2023) 46595–46623.

[47] M. Grusky, M. Naaman, Y. Artzi, Newsroom: A dataset of 1.3 million summaries with diverse extractive strategies, in: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers), 2018, pp. 708–719.

[48] J. Maynez, S. Narayan, B. Bohnet, R. McDonald, On faithfulness and factuality in abstractive summarization, in: Proceedings of the 58th annual meeting of the association for computational linguistics, 2020, pp. 1906–1919.

[49] W. Kryściński, B. McCann, C. Xiong, R. Socher, Evaluating the factual consistency of abstractive text summarization, in: Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP), 2020, pp. 9332–9346.

[50] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, P. Fung, Survey of hallucination in natural language generation, ACM computing surveys 55 (12) (2023) 1–38.

[51] J. R. Landis, G. G. Koch, The measurement of observer agreement for categorical data, biometrics (1977) 159–174.

[52] J. Demšar, Statistical comparisons of classifiers over multiple data sets, Journal of Machine learning research 7 (Jan) (2006) 1–30.

[53] J. Romano, J. D. Kromrey, J. Coraggio, J. Skowronek, Appropriate statistics for ordinal level data: Should we really be using t-test and cohen's d for evaluating group differences on the nsse and other surveys, in: Annual Meeting of the Florida Association of Institutional Research, 2006, pp. 1–33.

[54] N. Cliff, Dominance statistics: Ordinal analyses to answer ordinal questions., Psychological bulletin 114 (3) (1993) 494.

[55] P. Laban, T. Schnabel, P. N. Bennett, M. A. Hearst, Summac: Revisiting nli-based models for inconsistency detection in summarization,

Transactions of the Association for Computational Linguistics 10 (2022) 163–177.

[56] O. Honovich, R. Aharoni, J. Herzig, H. Taitelbaum, D. Kukliansy, V. Cohen, T. Scialom, I. Szpektor, A. Hassidim, Y. Matias, True: Re-evaluating factual consistency evaluation, in: Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2022, pp. 3905–3920.

[57] E. Durmus, H. He, M. Diab, Feqa: A question answering evaluation framework for faithfulness assessment in abstractive summarization, in: Proceedings of the 58th annual meeting of the association for computational linguistics, 2020, pp. 5055–5070.

[58] D. L. Davies, D. W. Bouldin, A cluster separation measure, IEEE Transactions on Pattern Analysis and Machine Intelligence PAMI-1 (2) (1979) 224–227.

[59] T. Caliński, J. Harabasz, A dendrite method for cluster analysis, Communications in Statistics-theory and Methods 3 (1) (1974) 1–27.

[60] P. J. Rousseeuw, Silhouettes: a graphical aid to the interpretation and validation of cluster analysis, Journal of computational and applied mathematics 20 (1987) 53–65.

[61] H. Schütze, C. D. Manning, P. Raghavan, Introduction to information retrieval, Vol. 39, Cambridge University Press Cambridge, 2008.

[62] M. Steinbach, G. Karypis, V. Kumar, A comparison of document clustering techniques, in: KDD Workshop on Text Mining, 2000, pp. 525–526.

[63] A. Strehl, J. Ghosh, Cluster ensembles—a knowledge reuse framework for combining multiple partitions, Journal of machine learning research 3 (Dec) (2002) 583–617.

[64] E. Amigó, J. Gonzalo, J. Artiles, F. Verdejo, A comparison of extrinsic clustering evaluation metrics based on formal constraints, Information retrieval 12 (4) (2009) 461–486.

Appendix A. Implementation Settings

Section 3 states what each stage does and why. This appendix collects the exact settings, so the method section carries the logic and the numbers stay checkable. Stage 1 fine-tunes a RoBERTa multi-label head; confident-learning correction against the verified anchor is gated by anchor confidence ≥ 0.70 and LLM probability ≤ 0.20, with a 3×3 sensitivity grid over $(0.65, 0.70, 0.75) \times (0.15, 0.20, 0.25)$ reported in Section 5.1. The `compatibility` stratum is seeded with 200 template-generated synthetic reviews (device, OS, and screen templates) plus 100 keyword-mined RRGen reviews, all human-checked. Stage 2 embeds with Sentence-BERT, reduces with UMAP, and clusters with HDBSCAN, aspect-first, producing 605 subclusters of mean size 15.9 against 194 flat clusters of mean size 471.0, over an 8,404-review sample rather than the full corpus; PageRank runs on the bipartite review-aspect graph. Stage 3 routes to one of five templates and re-prompts once with dimension-level feedback on any rubric failure. Stage 4 retrieves over a 15,100-document ChromaDB index built from five sources, with the IssueSpec injected per query rather than pre-indexed. Stage 5 trains five policies on distilGPT2 (82M parameters) from 400 SFT samples, 296 KTO pairs, and 100 DPO pairs; the Lagrangian loop runs 30 steps at batch size 4 with KL coefficient 0.05, and the compliance threshold sweep covers $\tau \in \{0.0, 0.5, 0.7, 0.9, 1.0\}$. SpecCov counts substantive-token overlap with the source cluster, tokens of length ≥ 5 after stop-word filtering, with quoted-string and coverage bonuses.

Appendix B. Annotator Guidelines

The lead author (PhD in software engineering with prior mobile-app review-mining experience) served as primary annotator across all five annotation tasks. The 490-review classification gold was additionally annotated by two further independent human raters on the identical item set, aligned by review id; all three first completed a shared 20-review calibration round, disagreements were adjudicated before the main task opened, and the released gold label is the majority vote over the three raters. All human annotation in this work is carried out by those same three people. For Stage 4, all three raters scored all 200 rows of the reported re-run independently and blind to condition; the discarded 400-row round had the lead author as primary rater with one additional rater on a 30-pair subset, and is described here only because the paper reports its withdrawal.

Separately, two Qwen2.5-3B-Instruct prompts (concise role-based and chain-of-thought) served as the second and third raters in the 99-review cluster-purity Krippendorff α calibration, which is an LLM-assisted panel and not one of the human rounds. For the 5,230-record verified anchor, which covers 5,229 distinct corpus rows, annotators classified each review into one of seven Maalej-Nabil classes (`bug_report`, `feature_request`, `performance`, `usability`, `compatibility`, `praise`, `other`) with multi-label assignment permitted when a review carried multiple concerns; reviews below the "fairly confident" threshold were skipped and excluded from the anchor. For the 490-review classification gold standard, a stratified sample of 70 reviews per class was drawn from the working corpus, and the lead author re-labeled each review independently and blinded to the V2 LLM prediction; the 490 gold labels serve as the held-out evaluation set for Cohen's κ progression (Table 12) and per-class F1 (Table 13). For the 5-dimension IssueSpec rubric (all 100 specs; an earlier stratified $n = 28$ subsample, 6 per issue type plus 2 extras), each spec was scored on a 1–5 Likert scale across five dimensions: *completeness* (are all required template fields filled?), *accuracy* (do field values correctly reflect the source cluster?), *actionability* (would a developer know what to do?), *specificity* (does the spec name concrete entities rather than generic terms?), and *clarity* (is the spec easy to read and unambiguous?), with each dimension scored independently. For the reported Stage 4 evaluation (100 reviews $\times$ 2 conditions, the same model with and without the IssueSpec, 200 rows), all three raters rated *quality* (1–5), *specificity* (1–5), and *helpful* (Y/N) while blinded to the generation method; the superseded 400-row round used four template-composed conditions and is retained in the release for comparison. In both rounds condition labels were replaced with random A/B/C/D codes and unsealed only at aggregation, with the assignment persisted as spreadsheets with JSON master keys for audit. A second human rater independently scored a 30-pair (`no_spec`, `full`) subset for inter-rater calibration. For the Y/P/N cluster-purity rubric (initial 50-cluster audit; 99-cluster three-rater subsample), annotators read 5 representative reviews per cluster and assigned Y (all 5 share the same sub-theme within the cluster's aspect), P (3 or 4 of 5 share the sub-theme), or N (fewer than 3 share); weighted purity is $\text{purity}_w = (|Y| + 0.5|P|)/(|Y| + |P| + |N|)$, with Y contributing 1.0, P contributing 0.5, and N contributing 0. All comparative evaluations were blinded throughout the rating window and unsealed only at aggregation; reviews or specs below the annotator confidence threshold were marked as "skip" and excluded from downstream analysis.

Table 6: Annotator-vs-LLM and inter-annotator deviation. The $\approx 25\%$ deviation is replicated on two independent strata; the κ-progression reaches moderate agreement [51]; the V5 endorsement row is a consistency check, not independent validation, since V5 trained on those corrections; LLM-judge calibration Δ bounds judge bias.

Annotator pair	sample	agreement / deviation
Lead-author vs V2 LLM	10,271	75% / **25%** mislabeled
Lead vs V2 / cleanlab / V5	490 gold	κ: 0.163 $\to$ 0.333 $\to$ **0.592**
3 human raters, Y/N verdict	490 gold	Fleiss κ = **0.968**; 479/490 unanimous on the verdict, 476/489 on the label
3 human raters, 7-class label	489 gold	Fleiss κ = **0.979**; $\alpha = 0.979$
V2 LLM vs V5; cleanlab vs V5	215,583	71.96% / 86.77% agree
cleanlab vs V5 (consistency, not independent)	40,291	88.66% support
V5 vs gold, strictly held out	307	$\kappa = \mathbf{0.616}$
3 human raters (Stage 4 quality)	400 rated rows	weighted κ 0.970–0.986; within-1 100%
3 human raters (Stage 4 helpful)	400 rated rows	κ 0.678–0.836; $\alpha = 0.781$
3-rater *LLM-assisted* purity check (lead + 2 Qwen prompts)	99 reviews	Krippendorff $\alpha = 0.451$
Lead vs Qwen-judge (purity, rubric)	50 / 28	$\Delta = -0.035$ / $+0.23$

Table 7: What the experiments establish, in one line each. Negative findings are marked, and they are findings rather than caveats.

	finding	evidence
F1	Type routing, not the language model, is what produces tracker-ready fields: removing it collapses strict fill from 0.96 to 0.69 and the per-type structural metrics to zero.	§5.1, A3
F2	*Corrected.* We reported that LLM annotation noise concentrates on minority classes. Per stratum it does the opposite: 1,302 of 1,307 disagreements are in the largest class. What holds is that the labeller omits two classes entirely, which is the failure a verified anchor repairs.	§5.1
F3	*Negative, and it overturns our own earlier claim.* With the generator held fixed, adding the typed IR changes response quality by $+0.04$ ($p = 0.38$); the $+2.35$ we previously reported was an artifact of comparing two different text composers.	§5.2
F4	Generator capability shows up mostly as whether fields get filled, not as the quality of the text once filled: template-fill spans 0.96–0.48 across four models while rubric quality spans 4.09–3.77.	§5.1
F5	*Negative.* The knowledge graph loses to count-matched flat clustering on all three intrinsic metrics; its defensible role is centrality-based prioritization.	§5.4, A1b
F6	*Negative.* Extractive coverage does not measure faithfulness: SpecCov shows no rank correlation with human judgement and orders generators backwards.	§5.1
F7	*Negative, then partial.* A CMDP constraint cannot be tested on a policy incapable of violating it; with one that can, the constraint binds and λ grows, but no trade-off is demonstrated.	§5.3
F8	Run-to-run generation noise is about 0.03 on template-fill, which is the yardstick separating the differences we claim from the ones we do not.	§6.4

Table 8: Cross-family five-dimension rubric, same Qwen2.5-3B judge as Table 10, on matched clusters. Panel A is the four-way comparison on clusters covered by every generator (bug reports only). Panel B widens the Claude-versus-Llama comparison to all five issue types. The Panel-A ordering matches the template-fill ordering in Table 9, but none of its six pairwise differences is significant at $n = 14$ (Wilcoxon p from 0.18 to 0.75), so read it as descriptive. Panel B, with twice the sample and all five issue types, does separate: +0.38 for Claude over Llama, Wilcoxon $p = 0.015$.

generator	n	mean	compl.	accur.	action.	spec.	clarity
Panel A, four-way, 14 matched bug-report clusters							
Claude (Anthropic)	14	**4.09**	4.00	4.50	3.71	4.36	3.86
Llama-3.3-70B (Meta)	14	3.96	3.86	4.50	4.07	3.86	3.50
Qwen2.5-3B (Alibaba)	14	3.83	3.07	4.50	4.00	3.86	3.71
Qwen2.5-1.5B (Alibaba)	13	3.77	3.00	4.15	4.46	3.62	3.62
Panel B, Claude vs Llama, 28 clusters, all five issue types							
Claude (Anthropic)	28	**4.01**	3.86	4.54	3.75	4.29	3.64
Llama-3.3-70B (Meta)	28	3.64	3.39	4.46	3.14	3.82	3.36

Table 9: Multi-LLM evidence summary across three model families (Anthropic, Meta, Alibaba). Cross-family rank preservation across Stage 3 and Stage 4 is the structural-bias control; LLM-as-judge calibration Δ vs the lead author bounds judge bias.

Use	Model (family)	Result
Stage 3 generate	Claude (Anthropic) / Llama-3.3-70B (Meta) / Qwen-3B (Alibaba) / Qwen-1.5B (Alibaba)	0.96 / 0.80 / 0.74 / 0.48
Stage 4 generate	Claude / Qwen-3B	0.61 / 0.54
5-dim rubric judge	Qwen-3B, all 100 specs	3.94/5 (28-spec subsample: 3.89)
Cross-family rubric	Claude / Llama / Qwen-3B / Qwen-1.5B	4.09 / 3.96 / 3.83 / 3.77 (Table 8)
Y/P/N purity judge	Qwen-3B vs lead-author	0.625 vs 0.66; $\Delta = -0.035$

Table 10: Experiment 1 five-dimension rubric (Qwen-judge, 1–5 scale) on the *complete* spec set: all 100 taxonomy specs and all 20 lead-author reference specs, scored in one run so the two columns are directly comparable. The middle column repeats the earlier stratified 28-spec subsample under the same judge and rubric; the gap between it and the full set is +0.05, which is the subsample error. The LLM-as-judge protocol follows the calibration-against-reference convention of [44, 45, 46].

dimension	taxonomy, $n = 100$	taxonomy, $n = 28$	lead-author ref, $n = 20$
completeness	**3.72**	3.79	3.20
accuracy	**4.43**	4.39	4.10
actionability	**3.86**	3.71	2.85
specificity	**4.15**	4.07	4.05
clarity	**3.52**	3.50	3.05
overall mean	**3.94**	3.89	3.45
sd across specs	0.59	–	0.41

Table 11: Experiment 1 substantive template-fill (strict criteria). The 0.96 vs 0.53 LLM-vs-GitHub gap is coverage, not content quality; the strict-vs-very-strict sweep preserves ordering. The exact-zero cells in `bugs:repro` and `features:story` columns for (b), (c), (d), and (e) are structural: free-form LLM, raw concatenation, lead-author, and human-GitHub specs do not emit the template-specific reproduction-step list or As-a/I-want/So-that triple at all when not prompted for them, so the strict-fill rate is zero by construction.

condition	strict	bugs:repro	features:story
(a) LLM + taxonomy	**0.96**	**0.73**	**0.97**
(b) LLM free-form	0.69	0.00	0.00
(c) raw concat	0.34	0.00	0.00
(d) lead-author ref ($n = 20$)	0.69	0.00	0.00
(e) human GitHub (3 repos)	0.53	0.13	0.00

Table 12: Cohen's κ progression vs the 490-review expert gold. Three Landis-Koch [51] bands crossed; cleanlab correction follows the confident-learning procedure of [23]. The final column repeats the measurement against the majority-vote gold of the three-rater human panel, showing the trajectory is not an artifact of a single annotator.

classifier	n	Cohen's κ	Landis-Koch band	κ vs 3-rater gold
V2 LLM original	490	0.163	slight	0.165
cleanlab-corrected V2	490	0.333	fair	0.334
V5 classifier	490	**0.592**	**moderate**	**0.590**

Table 13: Per-class F1 against the 490-review expert gold. V2 LLM is blind to the two minority classes (compatibility, performance). The verified-anchor procedure with targeted augmentation recovers them.

class	V2 LLM	cleanlab-V2	**V5**
`bug_report`	0.42	0.45	**0.58**
`feature_request`	0.32	0.37	**0.57**
`performance`	0.00	0.47	**0.77**
`usability`	0.11	0.27	**0.55**
`compatibility`	0.00	0.00	**0.83**
`praise`	0.38	0.62	**0.68**
`other`	0.30	0.47	**0.60**
macro F1	0.22	0.38	**0.65**

Table 14: Stage-4 re-run, same model with and without the IssueSpec, 100 paired reviews, three raters, all 200 rows scored by each. All figures are unadjusted, so the gain column equals the difference of the two condition columns; slot-stratified values are in the released JSON and differ by at most 0.04. The pooled effect is null, and the disagreement between the lead author and the two independent raters is larger than the effect being measured.

rater	no_spec	full	gain	95% CI	Wilcoxon p
lead author	4.04	4.60	+0.56	$[+0.44, +0.68]$	2×10^{-11}
independent rater 1	3.87	3.58	−0.29	$[-0.56, -0.01]$	0.06
independent rater 2	3.57	3.42	−0.15	$[-0.33, +0.03]$	0.09
pooled	3.83	3.87	+**0.04**	$[-0.10, +0.18]$	**0.38**

Table 15: Stage 2 designs head-to-head, comparing Pure HDBSCAN (without UMAP), original flat UMAP+HDBSCAN, and KG-hierarchical. Intrinsic metrics (Davies-Bouldin [58], Calinski-Harabasz [59], silhouette [60]) follow standard IR/clustering convention [61, 62, 63]. The Y/P/N weighted-purity rubric is a discrete-coding instantiation of the extrinsic cluster-purity metric [64, 61]. The count-controlled flat baseline (agglomerative-605) is reported separately in the A1b ablation (§5.5).

metric	Pure HDB	Flat U+H	**KG hier.**
n_clusters	3 (deg.)	194	**605**
mean cluster size	∼16,933	471.0	**15.9**
Davies-Bouldin (↓)	n/a	*withdrawn*	2.24
Calinski-Harabasz (↑)	n/a	*withdrawn*	1.85
Y/P/N purity (lead-author)	n/a	0.66	queued
Y/P/N purity (Qwen-judge)	n/a	0.625	0.390

Table 16: Ablation summary. "Removing this hurts" is the only outcome that justifies a component. Three of the seven do not clear that bar, and we report them as findings.

id	what is removed	effect of removing it	verdict
A1	the KG, feeding flat clusters to Stage 3	Stage 3 quality unchanged	no effect
A1b	the KG, against flat clustering matched to the same 605 clusters	flat wins on all three intrinsic metrics	**against the KG**
A2	hierarchical sub-clustering	purity 0.625 flat vs 0.390 KG	**against the KG**
A3	taxonomy grounding, so the LLM writes free-form	strict fill $0.96 \to 0.69$; per-type structure to zero	**justifies routing**
A4	HITL-2 review of specs	no change to benchmark scores	no effect at benchmark time
A5	retrieval, keeping the IssueSpec	all automatic metrics move $< 1\%$	**against RAG**
A6	the IssueSpec, keeping retrieval	quality $3.87 \to 3.83$ (-0.04, $p = 0.38$)	**no measurable effect** (§5.2)
A7	single-stream RLHF feedback	not run	deferred